Code Reference: server.mjs

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	server.mjs
Purpose	Local backend server used by the builder for drafts, parsing, publishing, compiler execution, authentication checks, and file operations.
Lines	3,632
Size	147,116 bytes
Talks to	public website runtime, portfolio catalog, Cloudflare/AI layer, GitHub/release layer
Functions	141
Variables/constants	140
API mentions	21
Signals	566

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Imports, requires, and external dependencies

Item	Explanation
Line 1	import { createServer } from "node:http";
Line 2	import { execFile, spawn } from "node:child_process";
Line 3	import { createHash } from "node:crypto";
Line 4	import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Line 5	import os from "node:os";
Line 6	import path from "node:path";
Line 7	import { promisify } from "node:util";
Line 8	import { fileURLToPath } from "node:url";

API endpoints mentioned or called

Item	Explanation
/api/catalog	Line 3343. Loads project/catalog data for the builder.
/api/templates	Line 3353. Loads appearance template definitions.
/api/publish-target	Line 3358. Reads or writes the Git publishing target and authorization state.
/api/system-check	Line 3367. Checks local tool/system readiness.
/api/app-update	Line 3372. Checks or starts builder app update behavior.
/api/security-report	Line 3377. Returns security/download/auth reporting for the builder.
/api/code/tools	Line 3386. Checks compiler/runtime availability.
/api/portfolio-ai	Line 3402. Handles AI assistant questions.
/api/code/beautify	Line 3407. Formats source code for cleaner display and editing.
/api/code/save	Line 3422. Saves source code into the local compile workspace.

Item	Explanation
/api/code/compile	Line 3433. Compiles or runs a source file and returns terminal output.
/api/code/install-tools	Line 3444. Attempts to install missing compiler tools.
/api/publish-target	Line 3455. Reads or writes the Git publishing target and authorization state.
/api/install-git	Line 3465. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/app-update/install	Line 3479. Checks or starts builder app update behavior.
/api/github-authenticate	Line 3493. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/sync-from-publish-target	Line 3510. Reads or writes the Git publishing target and authorization state.
/api/save-draft	Line 3525. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.
/api/apply-catalog	Line 3525. Loads project/catalog data for the builder.
/api/apply-catalog	Line 3534. Loads project/catalog data for the builder.
/api/upload	Line 3577. Project-specific local API endpoint. Inspect server.mjs and the fetch caller for exact behavior.

Important implementation signals

Item	Explanation
Compiler or simulator at line 1	Part of the Compile Code workspace or language/tool detection. Evidence: import { createServer } from "node:http";
Process execution at line 2	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: import { execFile, spawn } from "node:child_process";
Compiler or simulator at line 3	Part of the Compile Code workspace or language/tool detection. Evidence: import { createHash } from "node:crypto";
File read at line 4	Reads local builder state, project files, website assets, or configuration. Evidence: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from "node:fs/promises";
Compiler or simulator at line 5	Part of the Compile Code workspace or language/tool detection. Evidence: import os from "node:os";
Compiler or simulator at line 6	Part of the Compile Code workspace or language/tool detection. Evidence: import path from "node:path";
Compiler or simulator at line 7	Part of the Compile Code workspace or language/tool detection. Evidence: import { promisify } from "node:util";
Compiler or simulator at line 8	Part of the Compile Code workspace or language/tool detection. Evidence: import { fileURLToPath } from "node:url";
Process execution at line 15	Runs Git, compilers, installers, or helper tools outside the JavaScript runtime. Evidence: const execFileAsync = promisify(execFile);
File read at line 16	Reads local builder state, project files, website assets, or configuration. Evidence: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
Browser DOM at line 34	Builder or website interface behavior in the browser/Electron renderer. Evidence: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
Security or auth at line 40	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
Security or auth at line 51	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "_headers",
Security or auth at line 57	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
Security or auth at line 58	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
Security or auth at line 59	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
Security or auth at line 60	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthExtendedThreshold = 3;
Security or auth at line 63	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: const publishAuthorizationHelp = [
Security or auth at line 65	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Open Publishing target, enter the repository URL, then click Authenticate target.",
Git command at line 66	Repository state, publishing, branch checks, or release automation. Evidence: "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write access to the selected Pages repository.",

Item	Explanation
Security or auth at line 67	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Authenticate target only verifies write access and remembers it for the matching repository and branch.",
Security or auth at line 68	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Load from target only imports compatible website files from the authenticated repository into the local builder workspace.",
Git command at line 72	Repository state, publishing, branch checks, or release automation. Evidence: process.env.GIT_EXE,
Git command at line 73	Repository state, publishing, branch checks, or release automation. Evidence: "git",
Git command at line 74	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\cmd\\git.exe",
Git command at line 75	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files\\Git\\bin\\git.exe",
Git command at line 76	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\cmd\\git.exe",
Git command at line 77	Repository state, publishing, branch checks, or release automation. Evidence: "C:\\Program Files (x86)\\Git\\bin\\git.exe",
Git command at line 78	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "cmd", "git.exe") : "",
Git command at line 79	Repository state, publishing, branch checks, or release automation. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "Git", "bin", "git.exe") : ""
Compiler or simulator at line 87	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["gcc"],
Compiler or simulator at line 97	Part of the Compile Code workspace or language/tool detection. Evidence: verilog: {
Compiler or simulator at line 100	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Verilog",
Compiler or simulator at line 101	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 102	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 104	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: {
Compiler or simulator at line 107	Part of the Compile Code workspace or language/tool detection. Evidence: label: "SystemVerilog",
Compiler or simulator at line 108	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["iverilog", "vvp"],
Compiler or simulator at line 109	Part of the Compile Code workspace or language/tool detection. Evidence: winget: ["Icarus.Verilog"]
Compiler or simulator at line 118	Part of the Compile Code workspace or language/tool detection. Evidence: java: {
Compiler or simulator at line 119	Part of the Compile Code workspace or language/tool detection. Evidence: defaultFile: "Main.java",
Compiler or simulator at line 120	Part of the Compile Code workspace or language/tool detection. Evidence: extensions: [".java"],
Compiler or simulator at line 121	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Java",
Compiler or simulator at line 122	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["javac", "java"],
Compiler or simulator at line 129	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["node"],
Compiler or simulator at line 132	Part of the Compile Code workspace or language/tool detection. Evidence: python: {
Compiler or simulator at line 135	Part of the Compile Code workspace or language/tool detection. Evidence: label: "Python",
Compiler or simulator at line 136	Part of the Compile Code workspace or language/tool detection. Evidence: primaryTools: ["python"],

Item	Explanation
Compiler or simulator at line 149	Part of the Compile Code workspace or language/tool detection. Evidence: gcc: [
Compiler or simulator at line 150	Part of the Compile Code workspace or language/tool detection. Evidence: "gcc",
Compiler or simulator at line 151	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "gcc.exe") : ""
Installer at line 156	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages", "BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe", "mingw64", "bin", "g++.exe") : ""
Compiler or simulator at line 161	Part of the Compile Code workspace or language/tool detection. Evidence: javac: [
Compiler or simulator at line 162	Part of the Compile Code workspace or language/tool detection. Evidence: "javac",
Compiler or simulator at line 163	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\javac.exe"
Compiler or simulator at line 165	Part of the Compile Code workspace or language/tool detection. Evidence: java: [
Compiler or simulator at line 166	Part of the Compile Code workspace or language/tool detection. Evidence: "java",
Compiler or simulator at line 167	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\Eclipse Adoptium\\jdk-21.0.11.10-hotspot\\bin\\java.exe"
Compiler or simulator at line 169	Part of the Compile Code workspace or language/tool detection. Evidence: node: [
Compiler or simulator at line 170	Part of the Compile Code workspace or language/tool detection. Evidence: /(?:^|[\\/])node(?:\.exe)?$/i.test(process.execPath || "") ? process.execPath : "",
Compiler or simulator at line 171	Part of the Compile Code workspace or language/tool detection. Evidence: "node",
Compiler or simulator at line 172	Part of the Compile Code workspace or language/tool detection. Evidence: "C:\\Program Files\\nodejs\\node.exe",
Compiler or simulator at line 173	Part of the Compile Code workspace or language/tool detection. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "nodejs", "node.exe") : ""
Compiler or simulator at line 175	Part of the Compile Code workspace or language/tool detection. Evidence: python: [process.env.PYTHON, "python", "py"],
Compiler or simulator at line 176	Part of the Compile Code workspace or language/tool detection. Evidence: iverilog: ["iverilog", "C:\\iverilog\\bin\\iverilog.exe"],
Compiler or simulator at line 177	Part of the Compile Code workspace or language/tool detection. Evidence: vvp: ["vvp", "C:\\iverilog\\bin\\vvp.exe"],
Installer at line 180	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\ADI\\LTspice\\LTspice.exe",
Installer at line 181	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\LTC\\LTspiceXVII\\XVIIx64.exe",
Installer at line 182	Windows setup, update, shortcut, or installation behavior. Evidence: "C:\\Program Files\\Analog Devices\\LTspice\\LTspice.exe",
Installer at line 183	Windows setup, update, shortcut, or installation behavior. Evidence: process.env.LOCALAPPDATA ? path.join(process.env.LOCALAPPDATA, "Programs", "ADI", "LTspice", "LTspice.exe") : ""
Local storage at line 186	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolCache = new Map();
Local storage at line 187	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: const compileToolVersionCache = new Map();
Cloudflare or AI at line 192	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: "Answer the visitor's question first in a concise ChatGPT-like format, then add portfolio links or context only when they help.",
Security or auth at line 210	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Do not invent portfolio projects, credentials, employers, files, or test results that are not in the context.",
Security or auth at line 218	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: function securityHeaders(extra = {}) {

Item	Explanation
Security or auth at line 222	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Security or auth at line 230	Publishing protection, API key safety, HTTP security, or user authorization. Evidence: ...securityHeaders(),
Local storage at line 232	Local persistence, draft state, auth cache, update snooze, or browser-side caching. Evidence: "Cache-Control": "no-store, no-cache, must-revalidate"
Compiler or simulator at line 263	Part of the Compile Code workspace or language/tool detection. Evidence: systemverilog: "systemverilog",
Compiler or simulator at line 264	Part of the Compile Code workspace or language/tool detection. Evidence: sv: "systemverilog",

Important constants and variables

Item	Explanation
root	Line 10: const root = path.dirname(fileURLToPath(import.meta.url));
portfolioRoot	Line 11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
compileRoot	Line 12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root), "compile-code"));
port	Line 13: const port = Number(process.env.PORT || 8080);
host	Line 14: const host = process.env.HOST || "0.0.0.0";
execFileAsync	Line 15: const execFileAsync = promisify(execFile);
packageJson	Line 16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() => "{}"));
types	Line 18: const types = {
draftPath	Line 38: const draftPath = path.join(root, "projects.local.json");
catalogPath	Line 39: const catalogPath = path.join(root, "projects.json");
publishAuthCachePath	Line 40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
publishPaths	Line 41: const publishPaths = [
publishAuthCacheTtlMs	Line 57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
publishAuthExtendedTtlMs	Line 58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
publishAuthHistoryWindowMs	Line 59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
publishAuthExtendedThreshold	Line 60: const publishAuthExtendedThreshold = 3;
defaultSiteRepository	Line 61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
blockedAppUpdateVersions	Line 62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
publishAuthorizationHelp	Line 63: const publishAuthorizationHelp = [
gitCandidates	Line 71: const gitCandidates = [
compileLanguageProfiles	Line 82: const compileLanguageProfiles = {
compileToolCandidates	Line 148: const compileToolCandidates = {
compileToolCache	Line 186: const compileToolCache = new Map();
compileToolVersionCache	Line 187: const compileToolVersionCache = new Map();
portfolioAiInstructions	Line 189: const portfolioAiInstructions = [
clean	Line 257: const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
aliases	Line 258: const aliases = {
text	Line 283: const text = String(source || "");
text	Line 289: const text = String(source || "");
ext	Line 295: const ext = path.extname(String(fileName || "").toLowerCase());

Item	Explanation
byName	Line 305: const byName = languageFromFileName(fileName, code);
source	Line 307: const source = String(code || "");
profile	Line 322: const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] || compileLanguageProfiles.javascript;
fallback	Line 323: const fallback = profile.defaultFile;
parsed	Line 324: const parsed = path.parse(String(value || fallback));
safeName	Line 325: const safeName = safeSegment(parsed.name, path.parse(fallback).name);
ext	Line 326: let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();
depth	Line 332: let depth = 0;
line	Line 337: const line = rawLine.trim();
formatted	Line 340: const formatted = `${" ".repeat(depth)}${line}`;
opens	Line 341: const opens = (line.match(/[{\[(]/g) || []).length;
closes	Line 342: const closes = (line.match(/[}\])]/g) || []).length;
normalized	Line 351: const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, " ");
lang	Line 352: const lang = normalizeCodeLanguage(language);
entries	Line 376: let entries = [];
fullPath	Line 383: const fullPath = path.join(folder, entry.name);
found	Line 388: const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
candidates	Line 396: const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
command	Line 406: const command = process.platform === "win32" ? "where" : "command";
args	Line 407: const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
result	Line 408: const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
found	Line 409: const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
found	Line 420: const found = await findExecutableUnder(
found	Line 430: const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`], 5);
elapsedSeconds	Line 444: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
elapsed	Line 445: const elapsed = elapsedSeconds ? ` in ${elapsedSeconds}s` : "";
status	Line 446: const status = result.timedOut
output	Line 449: const output = [result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n");
clean	Line 454: const clean = String(value || "");
output	Line 464: let output = String(text || "");
from	Line 466: const from = replacement?.from || "";
to	Line 467: const to = replacement?.to || "";
isCpp	Line 481: const isCpp = language === "cpp";
ext	Line 482: const ext = path.extname(String(fileName || "").toLowerCase());
header	Line 483: const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
parsed	Line 504: const parsed = path.parse(String(fileName || "program"));
result	Line 511: const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
version	Line 512: const version = [result.stdout, result.stderr]
elapsedSeconds	Line 524: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 525: const status = result.timedOut

Item	Explanation
output	Line 528: const output = [result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n");
raw	Line 533: const raw = [result.stdout, result.stderr].map((part) => String(part || "").trimEnd()).filter(Boolean).join("\n");
lines	Line 534: const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
finishLines	Line 535: const finishLines = [];
signalLines	Line 536: const signalLines = [];
elapsedSeconds	Line 544: const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
status	Line 545: const status = result.timedOut
body	Line 548: const body = signalLines.length
suffix	Line 555: const suffix = signalLines.length && finishLines.length ? `\n${finishLines.join("\n")}` : "";
timeoutMs	Line 560: const timeoutMs = options.timeoutMs || 20000;

JSON/YAML-style keys detected

Item	Explanation
.css	Line 19: ".css": "text/css",
.csv	Line 20: ".csv": "text/csv",
.html	Line 21: ".html": "text/html",
.ico	Line 22: ".ico": "image/x-icon",
.js	Line 23: ".js": "application/javascript",
.json	Line 24: ".json": "application/json",
.md	Line 25: ".md": "text/markdown",
.pdf	Line 26: ".pdf": "application/pdf",
.png	Line 27: ".png": "image/png",
.jpg	Line 28: ".jpg": "image/jpeg",
.jpeg	Line 29: ".jpeg": "image/jpeg",
.svg	Line 30: ".svg": "image/svg+xml",
.txt	Line 31: ".txt": "text/plain",
.webp	Line 32: ".webp": "image/webp",
.xml	Line 33: ".xml": "application/xml",
.xlsx	Line 34: ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
.zip	Line 35: ".zip": "application/zip"
g++	Line 153: "g++": [
clang++	Line 159: "clang++": ["clang++"],
X-Content-Type-Options	Line 220: "X-Content-Type-Options": "nosniff",
Referrer-Policy	Line 221: "Referrer-Policy": "strict-origin-when-cross-origin",
Permissions-Policy	Line 222: "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
Cross-Origin-Opener-Policy	Line 223: "Cross-Origin-Opener-Policy": "same-origin",
Content-Type	Line 231: "Content-Type": "application/json",
Cache-Control	Line 232: "Cache-Control": "no-store, no-cache, must-revalidate"
c++	Line 262: "c++": "cpp",
Accept	Line 1386: "Accept": accept,
User-Agent	Line 1387: "User-Agent": "OMB-Portfolio-Builder-AI"

Item	Explanation
Authorization	Line 3314: "Authorization": `Bearer ${apiKey}`,
Content-Type	Line 3315: "Content-Type": "application/json"
Content-Type	Line 3621: "Content-Type": types[path.extname(filePath)] || "application/octet-stream",
Cache-Control	Line 3622: "Cache-Control": "no-store, no-cache, must-revalidate"

Function-by-function detail

securityHeaders - lines 218-226

Item	Explanation
Source location	Lines 218-226 (9 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function securityHeaders(extra = {}) {
Touches	filesystem, authentication/security data
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
218: function securityHeaders(extra = {}) {
219:   return {
220:     "X-Content-Type-Options": "nosniff",
221:     "Referrer-Policy": "strict-origin-when-cross-origin",
222:     "Permissions-Policy": "camera=(), microphone=(), geolocation=(), payment=(), usb=()",
223:     "Cross-Origin-Opener-Policy": "same-origin",
224:     ...extra
225:   };
226: }
```

sendJson - lines 228-235

Item	Explanation
Source location	Lines 228-235 (8 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function sendJson(response, status, data) {
Touches	Mostly local calculations or local UI state.
Calls detected	writeHead, securityHeaders, end, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
228: function sendJson(response, status, data) {
229:   response.writeHead(status, {
230:     ...securityHeaders(),
231:     "Content-Type": "application/json",
232:     "Cache-Control": "no-store, no-cache, must-revalidate"
233:   });
234:   response.end(JSON.stringify(data, null, 2));
235: }
```

clampText - lines 237-239

Item	Explanation
Source location	Lines 237-239 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function clampText(value, maxLength = 12000) {
Touches	Mostly local calculations or local UI state.
Calls detected	slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
237: function clampText(value, maxLength = 12000) {
238:   return String(value || "").slice(0, maxLength);
239: }
```

stripHtmlToText - lines 241-254

Item	Explanation
Source location	Lines 241-254 (14 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function stripHtmlToText(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	replace, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
241: function stripHtmlToText(value = "") {
242:   return String(value || "")
243:     .replace(/<script[\s\S]*?<\/script>/gi, " ")
244:     .replace(/<style[\s\S]*?<\/style>/gi, " ")
245:     .replace(/<[^>]+>/g, " ")
246:     .replace(/ /gi, " ")
247:     .replace(/&amp;/gi, "&")
248:     .replace(/&lt;/gi, "<")
249:     .replace(/&gt;/gi, ">")
250:     .replace(/&quot;/gi, '"')
251:     .replace(/&#39;/g, "'")
252:     .replace(/\s+/g, " ")
253:     .trim();
254: }
```

normalizeCodeLanguage - lines 256-280

Item	Explanation
Source location	Lines 256-280 (25 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function normalizeCodeLanguage(value = "") {
Touches	filesystem

Item	Explanation
Calls detected	trim, toLowerCase, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
256: function normalizeCodeLanguage(value = "") {
257:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
258:   const aliases = {
259:     c: "c",
260:     cpp: "cpp",
261:     cplusplus: "cpp",
262:     "c++": "cpp",
263:     systemverilog: "systemverilog",
264:     sv: "systemverilog",
265:     verilog: "verilog",
266:     v: "verilog",
267:     ltspice: "ltspice",
268:     spice: "ltspice",
269:     cir: "ltspice",
270:     java: "java",
271:     javascript: "javascript",
272:     js: "javascript",
273:     node: "javascript",
274:     python: "python",
275:     py: "python",
276:     html: "html",
277:     htm: "html"
278:   };
279:   return aliases[clean] || (compileLanguageProfiles[clean] ? clean : "javascript");
280: }
```

sourceLooksCpp - lines 282-286

Item	Explanation
Source location	Lines 282-286 (5 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function sourceLooksCpp(source = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  282: function sourceLooksCpp(source = "") {
  283:   const text = String(source || "");
  284:   return
/#include\s*<(?:iostream|string|vector|array|map|unordered_map|memory|algorithm|optional|variant)>/.test(text)
||
  285:     /\b(std::|cout|cin|cerr|namespace|template\s*<|class\s+\w+|new\s+\w|delete\s+|constexpr|nullptr|using
\s+namespace)\b/.test(text);
  286: }
```

sourceLooksC - lines 288-292

Item	Explanation
Source location	Lines 288-292 (5 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.

Item	Explanation
Declaration	function sourceLooksC(source = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
288: function sourceLooksC(source = "") {
289:   const text = String(source || "");
290:   return /#include\s*<(?:stdio|stdlib|string|stdint|stdbool|math)\.h>/.test(text) ||
291:     /\b(printf|scanf|malloc|calloc|realloc|free|struct\s+\w+|typedef\s+struct)\b/.test(text);
292: }
```

languageFromFileName - lines 294-302

Item	Explanation
Source location	Lines 294-302 (9 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function languageFromFileName(fileName = "", code = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	extname, toLowerCase, sourceLooksCpp, sourceLooksC, entries, find, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
294: function languageFromFileName(fileName = "", code = "") {
295:   const ext = path.extname(String(fileName || "").toLowerCase());
296:   if (ext === ".h") {
297:     if (sourceLooksCpp(code)) return "cpp";
298:     if (sourceLooksC(code)) return "c";
299:     return "c";
300:   }
301:   return Object.entries(compileLanguageProfiles).find(([, profile]) =>
profile.extensions.includes(ext))?.[0] || "";
302: }
```

detectCodeLanguageFromSource - lines 304-319

Item	Explanation
Source location	Lines 304-319 (16 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function detectCodeLanguageFromSource(code = "", fileName = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	languageFromFileName, b, sourceLooksCpp, sourceLooksC
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 10 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  304: function detectCodeLanguageFromSource(code = "", fileName = "") {
  305:   const byName = languageFromFileName(fileName, code);
  306:   if (byName) return byName;
  307:   const source = String(code || "");
  308:   if (/<\/?[a-z][\s\S]*?>/i.test(source) || /<!doctype\s+html/i.test(source)) return "html";
  309:   if (/\b(always_ff|always_comb|always_latch|logic|interface|covergroup|assert\s+property|typedef\s+enum|
class\s+\w+)\b/.test(source)) return "systemverilog";
  310:   if (/\b(module|endmodule|always|assign|reg|wire|initial|posedge|negedge)\b/.test(source)) return
"verilog";
  311:   if (/^\s*\.?(tran|ac|dc|op|model|subckt|ends|param)\b/im.test(source) ||
/\bV\w+\s+\w+\s+\w+\s+(?:DC|SIN|PULSE)?/i.test(source)) return "ltspice";
  312:   if (/\b(def|elif|from\s+\w+\s+import|self|None|True|False)\b/.test(source)) return "python";
  313:   if (/\b(public\s+class|static\s+void\s+main|System\.out)\b/.test(source)) return "java";
  314:   if (sourceLooksCpp(source) || sourceLooksC(source) ||
/\b(#include|main\s*\(|puts\s*\(|fprintf|sizeof\s*\()\b/.test(source)) {
  315:     return sourceLooksCpp(source) ? "cpp" : "c";
  316:   }
  317:   if (/\b(const|let|var|function|=>|console\.log|document\.|window\.)\b/.test(source)) return
"javascript";
  318:   return "javascript";
  319: }
```

safeCodeFileName - lines 321-329

Item	Explanation
Source location	Lines 321-329 (9 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function safeCodeFileName(value = "", language = "javascript") {
Touches	filesystem
Calls detected	normalizeCodeLanguage, parse, safeSegment, extname, toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  321: function safeCodeFileName(value = "", language = "javascript") {
  322:   const profile = compileLanguageProfiles[normalizeCodeLanguage(language)] ||
compileLanguageProfiles.javascript;
  323:   const fallback = profile.defaultFile;
  324:   const parsed = path.parse(String(value || fallback));
  325:   const safeName = safeSegment(parsed.name, path.parse(fallback).name);
  326:   let ext = String(parsed.ext || path.extname(fallback)).toLowerCase();
  327:   if (!profile.extensions.includes(ext)) ext = path.extname(fallback);
  328:   return `${safeName}${ext}`;
  329: }
```

indentBraceCode - lines 331-345

Item	Explanation
Source location	Lines 331-345 (15 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function indentBraceCode(code = "") {
Touches	filesystem
Calls detected	replace, split, map, trim, max, repeat, match
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
331: function indentBraceCode(code = "") {
332:   let depth = 0;
333:   return String(code || "")
334:     .replace(/\r\n?/g, "\n")
335:     .split("\n")
336:     .map((rawLine) => {
337:       const line = rawLine.trim();
338:       if (!line) return "";
339:       if (/^[}\])]/.test(line)) depth = Math.max(0, depth - 1);
340:       const formatted = `${"  ".repeat(depth)}${line}`;
341:       const opens = (line.match(/[{\[(]/g) || []).length;
342:       const closes = (line.match(/[}\])]/g) || []).length;
343:       depth = Math.max(0, depth + opens - closes);
344:       return formatted;
345:     })
```

beautifyCode - lines 350-372

Item	Explanation
Source location	Lines 350-372 (23 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function beautifyCode(code = "", language = "javascript") {
Touches	filesystem
Calls detected	replace, normalizeCodeLanguage, includes, indentBraceCode, split, map, trim, join, trimEnd
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
350: function beautifyCode(code = "", language = "javascript") {
351:   const normalized = String(code || "").replace(/\r\n?/g, "\n").replace(/\t/g, "  ");
352:   const lang = normalizeCodeLanguage(language);
353:   if (["c", "cpp", "java", "javascript", "verilog", "systemverilog"].includes(lang)) {
354:     return indentBraceCode(normalized)
355:       .replace(/[ \t]+$/gm, "")
356:       .replace(/\n{4,}/g, "\n\n\n");
357:   }
358:   if (lang === "html") {
359:     return normalized
360:       .replace(/>\s+</g, ">\n<")
361:       .split("\n")
362:       .map((line) => line.trim())
363:       .join("\n")
364:       .trim();
365:   }
366:   return normalized
367:     .split("\n")
368:     .map((line) => line.trimEnd())
369:     .join("\n")
370:     .replace(/\n{4,}/g, "\n\n\n")
371:     .trimEnd();
372: }
```

findExecutableUnder - lines 374-392

Item	Explanation
Source location	Lines 374-392 (19 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function findExecutableUnder(folder, names = [], maxDepth = 5) {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	pathExists, readdir, join, isFile, some, toLowerCase, isDirectory
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  374: async function findExecutableUnder(folder, names = [], maxDepth = 5) {
  375:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return "";
  376:   let entries = [];
  377:   try {
  378:     entries = await readdir(folder, { withFileTypes: true });
  379:   } catch {
  380:     return "";
  381:   }
  382:   for (const entry of entries) {
  383:     const fullPath = path.join(folder, entry.name);
  384:     if (entry.isFile() && names.some((name) => entry.name.toLowerCase() === name.toLowerCase())) return
fullPath;
  385:   }
  386:   for (const entry of entries) {
  387:     if (!entry.isDirectory()) continue;
  388:     const found = await findExecutableUnder(path.join(folder, entry.name), names, maxDepth - 1);
  389:     if (found) return found;
  390:   }
  391:   return "";
  392: }
```

findTool - lines 394-437

Item	Explanation
Source location	Lines 394-437 (44 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function findTool(toolName) {
Touches	filesystem
Calls detected	has, get, filter, isAbsolute, pathExists, set, execFileAsync, split, map, trim, find, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  394: async function findTool(toolName) {
  395:   if (compileToolCache.has(toolName)) return compileToolCache.get(toolName);
  396:   const candidates = (compileToolCandidates[toolName] || [toolName]).filter(Boolean);
  397:   for (const candidate of candidates) {
  398:     if (path.isAbsolute(candidate)) {
  399:       if (await pathExists(candidate)) {
  400:         compileToolCache.set(toolName, candidate);
  401:         return candidate;
  402:       }
  403:       continue;
  404:     }
  405:     try {
  406:       const command = process.platform === "win32" ? "where" : "command";
  407:       const args = process.platform === "win32" ? [candidate] : ["-v", candidate];
  408:       const result = await execFileAsync(command, args, { timeout: 5000, windowsHide: true });
  409:       const found = String(result.stdout || "").split(/\r?\n/).map((line) => line.trim()).find(Boolean);
  410:       if (found) {
  411:         compileToolCache.set(toolName, found);
  412:         return found;
  413:       }
  414:     } catch {
  415:       // Try the next candidate.
  416:     }
```

```
  }

  if (["gcc", "g++"].includes(toolName) && process.env.LOCALAPPDATA) {
    const found = await findExecutableUnder(
      path.join(process.env.LOCALAPPDATA, "Microsoft", "WinGet", "Packages"),
      [toolName === "gcc" ? "gcc.exe" : "g++.exe"],
      7
    );
    compileToolCache.set(toolName, found || "");
    return found || "";
  }

  if (["javac", "java"].includes(toolName)) {
    const found = await findExecutableUnder("C:\\Program Files\\Eclipse Adoptium", [`${toolName}.exe`],
5);
    compileToolCache.set(toolName, found || "");
    return found || "";
  }

  compileToolCache.set(toolName, "");
  return "";
}
```

terminalLine - lines 439-441

Item	Explanation
Source location	Lines 439-441 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function terminalLine(label, text = "") {
Touches	filesystem
Calls detected	trim, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function terminalLine(label, text = "") {
  return [`$ ${label}`, String(text || "").trim()].filter(Boolean).join("\n");
}
```

processTerminalText - lines 443-451

Item	Explanation
Source location	Lines 443-451 (9 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function processTerminalText(result = {}) {
Touches	filesystem
Calls detected	isFinite, toFixed, map, trimEnd, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function processTerminalText(result = {}) {
  const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
  const elapsed = elapsedSeconds ? ` in ${elapsedSeconds}s` : "";
  const status = result.timedOut
    ? `Timed out after ${elapsedSeconds ? `${elapsedSeconds}s` : "the configured timeout"}`
    : `Exit code ${result.code ?? "unknown"}${elapsed}`;
  const output = [result.stdout, result.stderr].map((part) => String(part ||
```

```
"").trimEnd()).filter(Boolean).join("\n");
  450:   return [status, output || (result.ok ? "Completed without diagnostic output." : "No compiler output was
 returned.")].join("\n");
  451: }
```

pathVariantsForReplacement - lines 453-461

Item	Explanation
Source location	Lines 453-461 (9 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function pathVariantsForReplacement(value = "") {
Touches	filesystem
Calls detected	from, Set, normalize, replace, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
453: function pathVariantsForReplacement(value = "") {
454:   const clean = String(value || "");
455:   return Array.from(new Set([
456:     clean,
457:     path.normalize(clean),
458:     clean.replace(/\\/g, "/"),
459:     path.normalize(clean).replace(/\\/g, "/")
460:   ].filter(Boolean)));
461: }
```

replacePathReferences - lines 463-474

Item	Explanation
Source location	Lines 463-474 (12 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function replacePathReferences(text = "", replacements = []) {
Touches	Mostly local calculations or local UI state.
Calls detected	pathVariantsForReplacement, split, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
463: function replacePathReferences(text = "", replacements = []) {
464:   let output = String(text || "");
465:   for (const replacement of replacements) {
466:     const from = replacement?.from || "";
467:     const to = replacement?.to || "";
468:     if (!from || !to) continue;
469:     for (const variant of pathVariantsForReplacement(from)) {
470:       output = output.split(variant).join(to);
471:     }
472:   }
473:   return output;
474: }
```

processTerminalTextWithPaths - lines 476-478

Item	Explanation
Source location	Lines 476-478 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function processTerminalTextWithPaths(result = {}, replacements = []) {
Touches	filesystem
Calls detected	replacePathReferences, processTerminalText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
476: function processTerminalTextWithPaths(result = {}, replacements = []) {
477:   return replacePathReferences(processTerminalText(result), replacements);
478: }
```

cFamilyCompileProfile - lines 480-501

Item	Explanation
Source location	Lines 480-501 (22 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function cFamilyCompileProfile(language = "c", fileName = "main.c") {
Touches	Mostly local calculations or local UI state.
Calls detected	extname, toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
480: function cFamilyCompileProfile(language = "c", fileName = "main.c") {
481:   const isCpp = language === "cpp";
482:   const ext = path.extname(String(fileName || "").toLowerCase());
483:   const header = [".h", ".hpp", ".hh", ".hxx"].includes(ext);
484:   return {
485:     compiler: isCpp ? "g++" : "gcc",
486:     standard: isCpp ? "-std=c++20" : "-std=c17",
487:     languageFlag: isCpp ? "c++" : "c",
488:     label: isCpp ? "C++" : "C",
489:     header,
490:     flags: [
491:       isCpp ? "-std=c++20" : "-std=c17",
492:       "-Wall",
493:       "-Wextra",
494:       "-Wpedantic",
495:       "-Wshadow",
496:       "-O0",
497:       "-g3",
498:       "-fdiagnostics-color=never"
499:     ]
500:   };
501: }
```

cFamilyBinaryName - lines 503-506

Item	Explanation
Source location	Lines 503-506 (4 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.

Item	Explanation
Declaration	function cFamilyBinaryName(fileName = "program") {
Touches	Mostly local calculations or local UI state.
Calls detected	parse, safeSegment
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
503: function cFamilyBinaryName(fileName = "program") {
504:   const parsed = path.parse(String(fileName || "program"));
505:   return `${safeSegment(parsed.name, "program")}.exe`;
506: }
```

toolVersionLine - lines 508-521

Item	Explanation
Source location	Lines 508-521 (14 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function toolVersionLine(toolPath = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	has, get, runProcess, map, trim, filter, join, split, find, basename, set
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
508: async function toolVersionLine(toolPath = "") {
509:   if (!toolPath) return "";
510:   if (compileToolVersionCache.has(toolPath)) return compileToolVersionCache.get(toolPath);
511:   const result = await runProcess(toolPath, ["--version"], { timeoutMs: 7000 });
512:   const version = [result.stdout, result.stderr]
513:     .map((part) => String(part || "").trim())
514:     .filter(Boolean)
515:     .join("\n")
516:     .split(/\r?\n/)
517:     .map((line) => line.trim())
518:     .find(Boolean) || path.basename(toolPath);
519:   compileToolVersionCache.set(toolPath, version);
520:   return version;
521: }
```

cFamilyRunOutput - lines 523-530

Item	Explanation
Source location	Lines 523-530 (8 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function cFamilyRunOutput(result = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	isFinite, toFixed, map, trimEnd, filter, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  523: function cFamilyRunOutput(result = {}) {
  524:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
  525:   const status = result.timedOut
  526:     ? `Program timed out after ${elapsedSeconds ? `${elapsedSeconds}s` : "the configured timeout"}`
  527:     : `Program exit code ${result.code ?? "unknown"}${elapsedSeconds ? ` in ${elapsedSeconds}s` : ""}`;
  528:   const output = [result.stdout, result.stderr].map((part) => String(part ||
"").trimEnd()).filter(Boolean).join("\n");
  529:   return [status, output || (result.ok ? "Program completed without output." : "No program output was
returned.")].join("\n");
  530: }
```

cleanHdlSimulationOutput - lines 532-557

Item	Explanation
Source location	Lines 532-557 (26 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function cleanHdlSimulationOutput(result = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	map, trimEnd, filter, join, split, push, replace, isFinite, toFixed
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  532: function cleanHdlSimulationOutput(result = {}) {
  533:   const raw = [result.stdout, result.stderr].map((part) => String(part ||
"").trimEnd()).filter(Boolean).join("\n");
  534:   const lines = raw.split(/\r?\n/).map((line) => line.trimEnd()).filter(Boolean);
  535:   const finishLines = [];
  536:   const signalLines = [];
  537:   for (const line of lines) {
  538:     if (/\$finish called at/i.test(line)) {
  539:       finishLines.push(line.replace(/^.*?:\d+:\s*/, ""));
  540:     } else {
  541:       signalLines.push(line);
  542:     }
  543:   }
  544:   const elapsedSeconds = Number.isFinite(result.elapsedMs) ? (result.elapsedMs / 1000).toFixed(2) : "";
  545:   const status = result.timedOut
  546:     ? `Simulation timed out after ${elapsedSeconds ? `${elapsedSeconds}s` : "the configured timeout"}`
  547:     : `Simulation exit code ${result.code ?? "unknown"}${elapsedSeconds ? ` in ${elapsedSeconds}s` :
""}`;
  548:   const body = signalLines.length
  549:     ? signalLines.join("\n")
  550:     : finishLines.length
  551:       ? finishLines.join("\n")
  552:       : result.ok
  553:         ? "Simulation completed without printed output."
  554:         : "No simulator output was returned.";
  555:   const suffix = signalLines.length && finishLines.length ? `\n${finishLines.join("\n")}` : "";
  556:   return `${status}\n${body}${suffix}`;
  557: }
```

runProcess - lines 559-599

Item	Explanation
Source location	Lines 559-599 (41 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function runProcess(command, args = [], options = {}) {

Item	Explanation
Touches	filesystem, external processes
Calls detected	now, spawn, execFile, kill, on, toString, call, write, end, resolve, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
559: async function runProcess(command, args = [], options = {}) {
560:   const timeoutMs = options.timeoutMs || 20000;
561:   const cwd = options.cwd || compileRoot;
562:   return new Promise((resolve) => {
563:     const startedAt = Date.now();
564:     const child = spawn(command, args, {
565:       cwd,
566:       env: { ...process.env, ...(options.env || {}) },
567:       shell: false,
568:       windowsHide: true
569:     });
570:     let stdout = "";
571:     let stderr = "";
572:     let timedOut = false;
573:     const timer = setTimeout(() => {
574:       timedOut = true;
575:       if (process.platform === "win32" && child.pid) {
576:         execFile("taskkill", ["/pid", String(child.pid), "/T", "/F"], { windowsHide: true }, () => {});
577:       }
578:       child.kill("SIGKILL");
579:     }, timeoutMs);
580:     child.stdout?.on("data", (chunk) => {
581:       stdout += chunk.toString();
582:     });
583:     child.stderr?.on("data", (chunk) => {
584:       stderr += chunk.toString();
585:     });
586:     if (Object.prototype.hasOwnProperty.call(options, "input")) {
587:       child.stdin?.write(String(options.input || ""));
588:     }
589:     child.stdin?.end();
590:     child.on("error", (error) => {
591:       clearTimeout(timer);
592:       resolve({ ok: false, code: -1, stdout, stderr: `${stderr}\n${error.message}`.trim(), timedOut,
elapsedMs: Date.now() - startedAt });
593:     });
594:     child.on("close", (code) => {
595:       clearTimeout(timer);
596:       resolve({ ok: !timedOut && code === 0, code, stdout, stderr, timedOut, elapsedMs: Date.now() -
startedAt });
597:     });
598:   });
599: }
```

compileToolStatus - lines 601-620

Item	Explanation
Source location	Lines 601-620 (20 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function compileToolStatus() {
Touches	Mostly local calculations or local UI state.
Calls detected	Set, values, flatMap, findTool, entries, every, fromEntries, map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
async function compileToolStatus() {
  const tools = {};
  const uniqueTools = [...new Set(Object.values(compileLanguageProfiles).flatMap((profile) => profile.primaryTools))];
  for (const tool of uniqueTools) {
    tools[tool] = await findTool(tool);
  }
  const languages = {};
  for (const [id, profile] of Object.entries(compileLanguageProfiles)) {
    const required = profile.primaryTools;
    languages[id] = {
      label: profile.label,
      defaultFile: profile.defaultFile,
      extensions: profile.extensions,
      ready: required.every((tool) => tools[tool]),
      tools: Object.fromEntries(required.map((tool) => [tool, tools[tool] || "missing"])),
      installHints: profile.winget || []
    };
  }
  return { compileRoot, languages, tools };
}
```

isHdlLanguage - lines 622-624

Item	Explanation
Source location	Lines 622-624 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function isHdlLanguage(language = "") {
Touches	filesystem
Calls detected	includes, normalizeCodeLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function isHdlLanguage(language = "") {
  return ["verilog", "systemverilog"].includes(normalizeCodeLanguage(language));
}
```

inferCompileFileRole - lines 626-633

Item	Explanation
Source location	Lines 626-633 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function inferCompileFileRole(fileName = "", code = "", language = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	isHdlLanguage, toLowerCase, b, tb
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
function inferCompileFileRole(fileName = "", code = "", language = "") {
  if (!isHdlLanguage(language)) return "source";
  const haystack = `${fileName}\n${code}`.toLowerCase();
  if (/\b(tb|testbench)\b|(^|[_-])tb([_-]|\.)|test[_-]?bench|\$dumpfile|\$dumpvars|module\s+tb[_a-z0-9]*/i.test(haystack)) {
    return "testbench";
  }
  return "design";
```

```
633: }
```

normalizeCompileFileRole - lines 635-639

Item	Explanation
Source location	Lines 635-639 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function normalizeCompileFileRole(value = "", language = "") {
Touches	filesystem
Calls detected	isHdlLanguage, trim, toLowerCase, replace, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
635: function normalizeCompileFileRole(value = "", language = "") {
636:   if (!isHdlLanguage(language)) return "source";
637:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "");
638:   return ["tb", "testbench", "bench"].includes(clean) ? "testbench" : "design";
639: }
```

saveCompileSource - lines 641-662

Item	Explanation
Source location	Lines 641-662 (22 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin = "" }) {
Touches	filesystem
Calls detected	normalizeCodeLanguage, detectCodeLanguageFromSource, safeSegment, safeCodeFileName, parse, resolveInsideCompileRoot, mkdir, writeFile, clampText, normalizeCompileFileRole, inferCompileFileRole, toISOString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
641: async function saveCompileSource({ projectId, fileId, title, fileName, language, role = "", code, stdin =
 "" }) {
642:   const lang = normalizeCodeLanguage(language || detectCodeLanguageFromSource(code, fileName));
643:   const projectFolder = safeSegment(projectId, "project");
644:   const codeFileName = safeCodeFileName(fileName, lang);
645:   const fileFolder = safeSegment(fileId || path.parse(codeFileName).name, path.parse(codeFileName).name);
646:   const folder = resolveInsideCompileRoot(projectFolder, fileFolder);
647:   const sourcePath = resolveInsideCompileRoot(projectFolder, fileFolder, codeFileName);
648:   await mkdir(folder, { recursive: true });
649:   await writeFile(sourcePath, String(code || ""), "utf8");
650:   if (stdin) await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "stdin.txt"),
String(stdin), "utf8");
651:   const metadata = {
652:     id: fileFolder,
653:     title: clampText(title || codeFileName, 180),
654:     fileName: codeFileName,
655:     language: lang,
656:     role: normalizeCompileFileRole(role || inferCompileFileRole(codeFileName, code, lang), lang),
657:     sourcePath,
658:     savedAt: new Date().toISOString()
659:   };
660:   await writeFile(resolveInsideCompileRoot(projectFolder, fileFolder, "compile-meta.json"),
`${JSON.stringify(metadata, null, 2)}\n`, "utf8");
661:   return metadata;
662: }
```

javaMainClassName - lines 664-670

Item	Explanation
Source location	Lines 664-670 (7 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function javaMainClassName(code = "", fileName = "Main.java") {
Touches	Mostly local calculations or local UI state.
Calls detected	match, parse
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
664: function javaMainClassName(code = "", fileName = "Main.java") {
665:   const source = String(code || "");
666:   const publicMatch = source.match(/\bpublic\s+class\s+([A-Za-z_$][\w$]*)/);
667:   if (publicMatch) return publicMatch[1];
668:   const classMatch = source.match(/\bclass\s+([A-Za-z_$][\w$]*)/);
669:   return classMatch?.[1] || path.parse(fileName).name || "Main";
670: }
```

compileCacheKey - lines 672-677

Item	Explanation
Source location	Lines 672-677 (6 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileCacheKey(parts = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	createHash, update, stringify, digest, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
672: function compileCacheKey(parts = {}) {
673:   return createHash("sha256")
674:     .update(JSON.stringify(parts))
675:     .digest("hex")
676:     .slice(0, 24);
677: }
```

compileCacheDirectory - lines 679-681

Item	Explanation
Source location	Lines 679-681 (3 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileCacheDirectory(projectId, language, key) {
Touches	Mostly local calculations or local UI state.
Calls detected	resolveInsideCompileRoot, safeSegment
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  679: function compileCacheDirectory(projectId, language, key) {
  680:   return resolveInsideCompileRoot(safeSegment(projectId, "project"), ".build-cache",
safeSegment(language, "language"), key);
  681: }
```

cachedBuildLine - lines 683-685

Item	Explanation
Source location	Lines 683-685 (3 source lines).
Purpose	Reads, writes, or validates cached state.
Declaration	function cachedBuildLine(type, outputPath) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  683: function cachedBuildLine(type, outputPath) {
  684:   return `${type} cache hit: using existing artifact\n${outputPath}`;
  685: }
```

hdlModuleNames - lines 687-689

Item	Explanation
Source location	Lines 687-689 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function hdlModuleNames(code = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	matchAll, map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  687: function hdlModuleNames(code = "") {
  688:   return [...String(code || "").matchAll(/\bmodule\s+([A-Za-z_$][\w$]*)/g)].map((match) => match[1]);
  689: }
```

hdlHasWaveDump - lines 691-694

Item	Explanation
Source location	Lines 691-694 (4 source lines).

Item	Explanation
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function hdlHasWaveDump(code = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  691: function hdlHasWaveDump(code = "") {
  692:   const source = String(code || "");
  693:   return /\$dumpfile\s*\(/.test(source) && /\$dumpvars\s*\(/.test(source);
  694: }
```

hdlFilesFromPayload - lines 696-727

Item	Explanation
Source location	Lines 696-727 (32 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") {
Touches	filesystem
Calls detected	isArray, Map, normalizeCodeLanguage, detectCodeLanguageFromSource, isHdlLanguage, trim, safeCodeFileName, safeSegment, parse, set, clampText, normalizeCompileFileRole
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  696: function hdlFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "verilog") {
  697:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
  698:   const byId = new Map();
  699:   const addFile = (item = {}, fallbackId = "") => {
  700:     const code = String(item.code || "");
  701:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
  702:     if (!isHdlLanguage(language) || !code.trim()) return;
  703:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language].defaultFile, language);
  704:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
  705:     byId.set(id, {
  706:       id,
  707:       title: clampText(item.title || fileName, 180),
  708:       fileName,
  709:       language,
  710:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
  711:       code
  712:     });
  713:   };
  714:   incoming.forEach((item, index) => addFile(item, `hdl-${index + 1}`));
  715:   addFile({
  716:     id: payload.fileId,
  717:     title: payload.title,
  718:     fileName: activeFileName,
  719:     language: activeLanguage,
  720:     role: payload.role,
  721:     code: payload.code
  722:   }, "active");
  723:   return [...byId.values()].sort((a, b) => {
  724:     if (a.role === b.role) return a.fileName.localeCompare(b.fileName);
  725:     return a.role === "testbench" ? 1 : -1;
```

```
726:   });
727: }
```

compileActionFromPayload - lines 729-733

Item	Explanation
Source location	Lines 729-733 (5 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileActionFromPayload(value = "", language = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, toLowerCase, replace, includes, isHdlLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
729: function compileActionFromPayload(value = "", language = "") {
730:   const clean = String(value || "").trim().toLowerCase().replace(/[_\s-]+/g, "-");
731:   if (["compile", "build", "run", "simulate"].includes(clean)) return clean;
732:   return isHdlLanguage(language) ? "simulate" : "run";
733: }
```

compileActionLabel - lines 735-740

Item	Explanation
Source location	Lines 735-740 (6 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileActionLabel(action = "run", language = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	isHdlLanguage
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
735: function compileActionLabel(action = "run", language = "") {
736:   if (action === "compile") return "Compile";
737:   if (action === "build") return "Build project";
738:   if (action === "simulate") return "Simulate";
739:   return isHdlLanguage(language) ? "Simulate" : "Run";
740: }
```

compileWorkspaceFilesFromPayload - lines 742-770

Item	Explanation
Source location	Lines 742-770 (29 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage = "javascript") {
Touches	filesystem
Calls detected	isArray, Map, trim, normalizeCodeLanguage, detectCodeLanguageFromSource, safeCodeFileName, safeSegment, parse, set, clampText, normalizeCompileFileRole, inferCompileFileRole

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  742: function compileWorkspaceFilesFromPayload(payload = {}, activeFileName = "", activeLanguage =
"javascript") {
  743:   const incoming = Array.isArray(payload.workspaceFiles) ? payload.workspaceFiles : [];
  744:   const byId = new Map();
  745:   const addFile = (item = {}, fallbackId = "") => {
  746:     const code = String(item.code || "");
  747:     if (!code.trim()) return;
  748:     const language = normalizeCodeLanguage(item.language || detectCodeLanguageFromSource(code,
item.fileName));
  749:     const fileName = safeCodeFileName(item.fileName || activeFileName ||
compileLanguageProfiles[language]?.defaultFile || "source.txt", language);
  750:     const id = safeSegment(item.id || fallbackId || fileName, path.parse(fileName).name);
  751:     byId.set(id, {
  752:       id,
  753:       title: clampText(item.title || fileName, 180),
  754:       fileName,
  755:       language,
  756:       role: normalizeCompileFileRole(item.role || inferCompileFileRole(fileName, code, language),
language),
  757:       code
  758:     });
  759:   };
  760:   incoming.forEach((item, index) => addFile(item, `workspace-${index + 1}`));
  761:   addFile({
  762:     id: payload.fileId,
  763:     title: payload.title,
  764:     fileName: activeFileName,
  765:     language: activeLanguage,
  766:     role: payload.role,
  767:     code: payload.code
  768:   }, "active");
  769:   return [...byId.values()].sort((a, b) => a.fileName.localeCompare(b.fileName));
  770: }
```

writeCompileWorkspaceSources - lines 772-788

Item	Explanation
Source location	Lines 772-788 (17 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {
Touches	filesystem
Calls detected	mkdir, Map, includes, normalizeCodeLanguage, extname, toLowerCase, parse, get, set, join, writeFile, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  772: async function writeCompileWorkspaceSources(files = [], targetDir = "", options = {}) {
  773:   await mkdir(targetDir, { recursive: true });
  774:   const seen = new Map();
  775:   const written = [];
  776:   for (const file of files) {
  777:     if (options.languages?.length && !options.languages.includes(normalizeCodeLanguage(file.language)))
continue;
  778:     if (options.extensions?.length &&
!options.extensions.includes(path.extname(file.fileName).toLowerCase())) continue;
  779:     const parsed = path.parse(file.fileName);
  780:     const count = seen.get(file.fileName) || 0;
  781:     seen.set(file.fileName, count + 1);
  782:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
  783:     const sourcePath = path.join(targetDir, uniqueName);
```

```
784:     await writeFile(sourcePath, file.code, "utf8");
785:     written.push({ ...file, uniqueName, sourcePath });
786:   }
787:   return written;
788: }
```

sourceDisplayName - lines 790-792

Item	Explanation
Source location	Lines 790-792 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function sourceDisplayName(file = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	basename
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
790: function sourceDisplayName(file = {}) {
791:   return file.uniqueName || file.fileName || path.basename(file.sourcePath || "source");
792: }
```

cFamilyWorkspaceSources - lines 794-803

Item	Explanation
Source location	Lines 794-803 (10 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	Set, filter, has, extname, toLowerCase, some, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
794: function cFamilyWorkspaceSources(files = [], language = "c", activeFileName = "") {
795:   const exts = language === "cpp"
796:     ? new Set([".cpp", ".cc", ".cxx", ".c++", ".hpp", ".hh", ".hxx", ".h"])
797:     : new Set([".c", ".h"]);
798:   const selected = files.filter((file) => exts.has(path.extname(file.fileName).toLowerCase()));
799:   if (!selected.some((file) => file.fileName === activeFileName)) {
800:     selected.push(...files.filter((file) => file.fileName === activeFileName));
801:   }
802:   return selected;
803: }
```

writeHdlSimulationSources - lines 805-821

Item	Explanation
Source location	Lines 805-821 (17 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function writeHdlSimulationSources(files = [], cacheDir = "") {

Item	Explanation
Touches	filesystem
Calls detected	join, rm, mkdir, Map, parse, get, set, writeFile, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
805: async function writeHdlSimulationSources(files = [], cacheDir = "") {
806:   const sourceDir = path.join(cacheDir, "sources");
807:   await rm(sourceDir, { recursive: true, force: true });
808:   await mkdir(sourceDir, { recursive: true });
809:   const seen = new Map();
810:   const written = [];
811:   for (const file of files) {
812:     const parsed = path.parse(file.fileName);
813:     const count = seen.get(file.fileName) || 0;
814:     seen.set(file.fileName, count + 1);
815:     const uniqueName = count ? `${parsed.name}_${count}${parsed.ext}` : file.fileName;
816:     const sourcePath = path.join(sourceDir, uniqueName);
817:     await writeFile(sourcePath, file.code, "utf8");
818:     written.push({ ...file, uniqueName, sourcePath });
819:   }
820:   return written;
821: }
```

findFilesByExtension - lines 823-836

Item	Explanation
Source location	Lines 823-836 (14 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) {
Touches	Mostly local calculations or local UI state.
Calls detected	pathExists, readdir, join, isDirectory, push, isFile, toLowerCase, endsWith
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
823: async function findFilesByExtension(folder = "", extension = ".vcd", maxDepth = 3) {
824:   if (!folder || maxDepth < 0 || !(await pathExists(folder))) return [];
825:   const entries = await readdir(folder, { withFileTypes: true }).catch(() => []);
826:   const matches = [];
827:   for (const entry of entries) {
828:     const fullPath = path.join(folder, entry.name);
829:     if (entry.isDirectory()) {
830:       matches.push(...await findFilesByExtension(fullPath, extension, maxDepth - 1));
831:     } else if (entry.isFile() && entry.name.toLowerCase().endsWith(extension.toLowerCase())) {
832:       matches.push(fullPath);
833:     }
834:   }
835:   return matches;
836: }
```

normalizeVcdValue - lines 838-844

Item	Explanation
Source location	Lines 838-844 (7 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.

Item	Explanation
Declaration	function normalizeVcdValue(raw = "") {
Touches	filesystem
Calls detected	trim, toLowerCase, slice, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
838: function normalizeVcdValue(raw = "") {
839:   const clean = String(raw || "").trim();
840:   if (!clean) return "x";
841:   if (/^[01xz]$/i.test(clean)) return clean.toLowerCase();
842:   if (/^[br][01xz_]+$/i.test(clean)) return clean.slice(1).replace(/_/g, "").toLowerCase();
843:   return clean.toLowerCase();
844: }
```

parseVcdScopeText - lines 846-933

Item	Explanation
Source location	Lines 846-933 (88 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function parseVcdScopeText(text = "", source = "waveform.vcd") {
Touches	filesystem, authentication/security data
Calls detected	split, Map, trim, startsWith, replace, push, includes, match, pop, has, set, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
846: function parseVcdScopeText(text = "", source = "waveform.vcd") {
847:   const lines = String(text || "").split(/\r?\n/);
848:   const scopes = [];
849:   const signalsByCode = new Map();
850:   const timeScaleParts = [];
851:   let inTimescale = false;
852:   let time = 0;
853:   let maxTime = 0;
854:   let definitionDone = false;
855:   for (const rawLine of lines) {
856:     const line = rawLine.trim();
857:     if (!line) continue;
858:     if (line.startsWith("$timescale")) {
859:       inTimescale = true;
860:       const inline = line.replace("$timescale", "").replace("$end", "").trim();
861:       if (inline) timeScaleParts.push(inline);
862:       continue;
863:     }
864:     if (inTimescale) {
865:       if (line === "$end") {
866:         inTimescale = false;
867:       } else {
868:         timeScaleParts.push(line.replace("$end", "").trim());
869:         if (line.includes("$end")) inTimescale = false;
870:       }
871:       continue;
872:     }
873:     const scopeMatch = line.match(/^\$scope\s+\S+\s+(.+?)\s+\$end$/);
874:     if (scopeMatch) {
875:       scopes.push(scopeMatch[1]);
876:       continue;
877:     }
878:     if (/^\$upscope\b/.test(line)) {
```

```
879:       scopes.pop();
880:       continue;
881:     }
882:     const varMatch = line.match(/^\$var\s+\S+\s+(\d+)\s+(\S+)\s+(.+?)\s+\$end$/);
883:     if (varMatch) {
884:       const [, width, code, reference] = varMatch;
885:       if (!signalsByCode.has(code) && signalsByCode.size < 64) {
886:         const cleanReference = reference.replace(/\s+\[[^\]]+\]$/, "");
887:         signalsByCode.set(code, {
888:           code,
889:           name: [...scopes, cleanReference].filter(Boolean).join("."),
890:           reference: cleanReference,
891:           width: Number(width) || 1,
892:           changes: []
893:         });
894:       }
895:       continue;
896:     }
897:     if (line.startsWith("$enddefinitions")) {
898:       definitionDone = true;
899:       continue;
900:     }
901:     if (!definitionDone) continue;
902:     if (line.startsWith("#")) {
903:       time = Number(line.slice(1)) || 0;
904:       maxTime = Math.max(maxTime, time);
905:       continue;
906:     }
907:     let code = "";
908:     let value = "";
909:     const vectorMatch = line.match(/^([br][01xz_]+)\s+(\S+)$/i);
910:     if (vectorMatch) {
911:       value = normalizeVcdValue(vectorMatch[1]);
912:       code = vectorMatch[2];
913:     } else if (/^[01xz]/i.test(line)) {
914:       value = normalizeVcdValue(line[0]);
915:       code = line.slice(1).trim();
916:     }
917:     const signal = signalsByCode.get(code);
918:     if (!signal || !value) continue;
919:     const previous = signal.changes[signal.changes.length - 1];
920:     if (!previous || previous.value !== value || previous.time !== time) {
921:       if (signal.changes.length < 600) signal.changes.push({ time, value });
922:     }
923:   }
924:   const signals = [...signalsByCode.values()]
925:     .filter((signal) => signal.changes.length)
926:     .slice(0, 32);
927:   return {
928:     source: path.basename(source),
929:     timeScale: timeScaleParts.join(" ").replace(/\s+/g, " ").trim() || "ticks",
930:     maxTime,
931:     signals
932:   };
933: }
```

readHdlWaveform - lines 935-942

Item	Explanation
Source location	Lines 935-942 (8 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function readHdlWaveform(cacheDir = "") {
Touches	filesystem, authentication/security data
Calls detected	findFilesByExtension, sort, readFile, slice, parseVcdScopeText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
935: async function readHdlWaveform(cacheDir = "") {
936:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
937:   if (!vcdFiles.length) return null;
938:   const selected = vcdFiles.sort((a, b) => a.length - b.length)[0];
939:   const text = await readFile(selected, "utf8");
940:   const limited = text.length > 6_000_000 ? text.slice(0, 6_000_000) : text;
941:   return parseVcdScopeText(limited, selected);
```

```
942: }
```

clearHdlWaveforms - lines 944-947

Item	Explanation
Source location	Lines 944-947 (4 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function clearHdlWaveforms(cacheDir = "") {
Touches	filesystem
Calls detected	findFilesByExtension, all, map, rm
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
944: async function clearHdlWaveforms(cacheDir = "") {
945:   const vcdFiles = await findFilesByExtension(cacheDir, ".vcd", 4);
946:   await Promise.all(vcdFiles.map((filePath) => rm(filePath, { force: true }).catch(() => {})));
947: }
```

compileAndRunCode - lines 949-1319

Item	Explanation
Source location	Lines 949-1319 (371 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function compileAndRunCode(payload = {}) {
Touches	filesystem, authentication/security data
Calls detected	trim, toLowerCase, normalizeCodeLanguage, detectCodeLanguageFromSource, safeCodeFileName, saveCompileSource, safeSegment, resolveInsideCompileRoot, compileActionFromPayload, compileWorkspaceFilesFromPayload, async, now
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	44 await expression(s), 15 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
949: async function compileAndRunCode(payload = {}) {
950:   const requestedLanguage = String(payload.language || "").trim().toLowerCase();
951:   const language = requestedLanguage && requestedLanguage !== "auto"
952:     ? normalizeCodeLanguage(requestedLanguage)
953:     : detectCodeLanguageFromSource(payload.code, payload.fileName);
954:   const profile = compileLanguageProfiles[language] || compileLanguageProfiles.javascript;
955:   const fileName = safeCodeFileName(payload.fileName, language);
956:   const saved = await saveCompileSource({ ...payload, language, fileName });
957:   const projectFolder = safeSegment(payload.projectId, "project");
958:   const sourcePath = resolveInsideCompileRoot(projectFolder, safeSegment(saved.id), saved.fileName);
959:   const sourceCode = String(payload.code || "");
960:   const action = compileActionFromPayload(payload.action, language);
961:   const allWorkspaceFiles = compileWorkspaceFilesFromPayload(payload, saved.fileName, language);
962:   let runDir = "";
963:   let runSourcePath = "";
964:   let runWorkspaceSources = [];
965:   const ensureRunSource = async (options = {}) => {
966:     if (runDir && runSourcePath) return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
967:     const runId = `${Date.now()}-${Math.random().toString(16).slice(2)}`;
968:     runDir = resolveInsideCompileRoot(projectFolder, ".runs", runId);
969:     await mkdir(runDir, { recursive: true });
970:     const filesToWrite = Array.isArray(options.files) && options.files.length ? options.files :
allWorkspaceFiles;
971:     runWorkspaceSources = await writeCompileWorkspaceSources(filesToWrite, runDir, {
972:       languages: options.languages,
```

```
      extensions: options.extensions
    });
    const active = runWorkspaceSources.find((file) => file.id === saved.id || file.fileName === saved.fileName);
    runSourcePath = active?.sourcePath || path.join(runDir, saved.fileName);
    if (!active) {
      await cp(sourcePath, runSourcePath, { force: true });
      runWorkspaceSources.push({
        id: saved.id,
        title: saved.title,
        fileName: saved.fileName,
        language,
        role: saved.role,
        code: sourceCode,
        uniqueName: saved.fileName,
        sourcePath: runSourcePath
      });
    }
    return { runDir, runSourcePath, writtenSources: runWorkspaceSources };
  };
  const stdin = String(payload.stdin || "");
  const forceRebuild = payload.forceRebuild === true;
  const terminal = [];
  terminal.push(`${compileActionLabel(action, language)} request for ${saved.fileName}`);
  const append = (label, result) => {
    terminal.push(terminalLine(label, processTerminalText(result)));
  };
  const missing = async (tools) => {
    const found = {};
    for (const tool of tools) found[tool] = await findTool(tool);
    const missingTools = tools.filter((tool) => !found[tool]);
    return { found, missingTools };
  };

  if (language === "html") {
    const openTags = (String(payload.code || "").match(/<([a-z][\w:-]*)\b[^>]*>/gi) || []).length;
    const closeTags = (String(payload.code || "").match(/<\/([a-z][\w:-]*)>/gi) || []).length;
    const ok = openTags >= closeTags;
    return {
      ok,
      language,
      saved,
      terminal: [
        "HTML validation pass",
        ok ? "No obvious tag-balance issue was detected." : "The file may have more closing tags than opening tags.",
        `Saved source: ${saved.sourcePath}`
      ].join("\n")
    };
  }

  const stdinOptions = stdin ? { input: stdin } : {};
  let ok = false;

  if (language === "javascript") {
    const node = await findTool("node");
    if (!node) return { ok: false, language, saved, terminal: "Node.js was not found. Install Node.js to run JavaScript." };
    const jsFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "javascript");
    const runSource = await ensureRunSource({ files: jsFiles.length ? jsFiles : allWorkspaceFiles, languages: ["javascript"] });
    const targets = action === "build"
      ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "javascript")
      : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
    ok = true;
    for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName: saved.fileName }]) {
      const check = await runProcess(node, ["--check", target.sourcePath], { cwd: runSource.runDir, timeoutMs: 15000 });
      terminal.push(terminalLine(`${path.basename(node)} --check ${sourceDisplayName(target)}`, processTerminalText(check)));
      ok = ok && check.ok;
    }
    if (action === "compile" || action === "build") {
      terminal.push(ok ? "JavaScript syntax check completed." : "JavaScript syntax check found errors.");
    } else if (ok) {
      const run = await runProcess(node, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs: 20000, ...stdinOptions });
      append(`${path.basename(node)} ${saved.fileName}`, run);
      ok = run.ok;
    }
  } else if (language === "python") {
    const python = await findTool("python");
    if (!python) return { ok: false, language, saved, terminal: "Python was not found. Install Python to run Python code." };
    const pyFiles = allWorkspaceFiles.filter((file) => normalizeCodeLanguage(file.language) === "python");
    const runSource = await ensureRunSource({ files: pyFiles.length ? pyFiles : allWorkspaceFiles, languages: ["python"] });
    const targets = action === "build"
      ? runSource.writtenSources.filter((file) => normalizeCodeLanguage(file.language) === "python")
```

```
 1053:       : runSource.writtenSources.filter((file) => file.sourcePath === runSource.runSourcePath);
 1054:     ok = true;
 1055:     for (const target of targets.length ? targets : [{ sourcePath: runSource.runSourcePath, uniqueName:
saved.fileName }]) {
 1056:       const check = await runProcess(python, ["-m", "py_compile", target.sourcePath], { cwd:
runSource.runDir, timeoutMs: 15000 });
 1057:       terminal.push(terminalLine(`${path.basename(python)} -m py_compile ${sourceDisplayName(target)}`,
processTerminalText(check)));
 1058:       ok = ok && check.ok;
 1059:     }
 1060:     if (action === "compile" || action === "build") {
 1061:       terminal.push(ok ? "Python bytecode compile check completed." : "Python compile check found
errors.");
 1062:     } else if (ok) {
 1063:       const run = await runProcess(python, [runSource.runSourcePath], { cwd: runSource.runDir, timeoutMs:
 20000, ...stdinOptions });
 1064:       append(`${path.basename(python)} ${saved.fileName}`, run);
 1065:       ok = run.ok;
 1066:     }
 1067:   } else if (language === "c" || language === "cpp") {
 1068:     const cProfile = cFamilyCompileProfile(language, saved.fileName);
      ... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

append - lines 996-998

Item	Explanation
Source location	Lines 996-998 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const append = (label, result) => {
Touches	filesystem
Calls detected	push, terminalLine, processTerminalText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 996:   const append = (label, result) => {
 997:     terminal.push(terminalLine(label, processTerminalText(result)));
 998:   };
```

missing - lines 999-1004

Item	Explanation
Source location	Lines 999-1004 (6 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const missing = async (tools) => {
Touches	Mostly local calculations or local UI state.
Calls detected	async, findTool, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 999:   const missing = async (tools) => {
1000:     const found = {};
1001:     for (const tool of tools) found[tool] = await findTool(tool);
1002:     const missingTools = tools.filter((tool) => !found[tool]);
1003:     return { found, missingTools };
1004:   };
```

installCompilerTools - lines 1321-1352

Item	Explanation
Source location	Lines 1321-1352 (32 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	async function installCompilerTools(language = "") {
Touches	filesystem
Calls detected	normalizeCodeLanguage, findTool, runProcess, push, terminalLine, join, processTerminalText, clear, compileToolStatus
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1321: async function installCompilerTools(language = "") {
 1322:   const lang = normalizeCodeLanguage(language);
 1323:   const profile = compileLanguageProfiles[lang] || compileLanguageProfiles.javascript;
 1324:   if (!profile.winget?.length) {
 1325:     return {
 1326:       ok: false,
 1327:       language: lang,
 1328:       terminal: `${profile.label} does not have an automatic Winget compiler install configured. Install
the required tools manually, then restart the builder.`
 1329:     };
 1330:   }
 1331:   const winget = await findTool("winget") || "winget";
 1332:   const terminal = [];
 1333:   let ok = true;
 1334:   for (const packageId of profile.winget) {
 1335:     const args = [
 1336:       "install",
 1337:       "--source",
 1338:       "winget",
 1339:       "--accept-source-agreements",
 1340:       "--accept-package-agreements",
 1341:       "--silent",
 1342:       "--id",
 1343:       packageId,
 1344:       "--exact"
 1345:     ];
 1346:     const result = await runProcess(winget, args, { cwd: root, timeoutMs: 600000 });
 1347:     terminal.push(terminalLine(`winget ${args.join(" ")}`, processTerminalText(result)));
 1348:     ok = ok && result.ok;
 1349:   }
 1350:   compileToolCache.clear();
 1351:   return { ok, language: lang, terminal: terminal.join("\n\n"), tools: await compileToolStatus() };
 1352: }
```

sourceLooksTextual - lines 1354-1360

Item	Explanation
Source location	Lines 1354-1360 (7 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function sourceLooksTextual(source = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1354: function sourceLooksTextual(source = {}) {
 1355:   const url = String(source.url || "");
 1356:   const kind = String(source.kind || "").toLowerCase();
 1357:   if (["text", "webpage", "github", "linkedin", "public_profile", "resume_text", "code"].includes(kind))
return true;
 1358:   return /\.(txt|md|markdown|csv|json|xml|log|c|h|cpp|hpp|py|js|mjs|ts|tsx|v|sv|vhdl?|spice|cir|net|asc|s
ch|kicad_sch|kicad_pcb|ino|xdc|sdc|tcl)$/i.test(url)
 1359:     || /github\.com|raw\.githubusercontent\.com/i.test(url);
 1360: }
```

sourceUrlAllowed - lines 1362-1379

Item	Explanation
Source location	Lines 1362-1379 (18 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function sourceUrlAllowed(url) {
Touches	Mostly local calculations or local UI state.
Calls detected	URL, toLowerCase, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1362: function sourceUrlAllowed(url) {
1363:   try {
1364:     const parsed = new URL(url);
1365:     const hostName = parsed.hostname.toLowerCase();
1366:     return [
1367:       "localhost",
1368:       "127.0.0.1",
1369:       "github.com",
1370:       "api.github.com",
1371:       "raw.githubusercontent.com",
1372:       "gist.githubusercontent.com",
1373:       "linkedin.com",
1374:       "www.linkedin.com"
1375:     ].includes(hostName);
1376:   } catch {
1377:     return false;
1378:   }
1379: }
```

gitHubHeaders - lines 1384-1389

Item	Explanation
Source location	Lines 1384-1389 (6 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function gitHubHeaders(accept = "application/vnd.github+json") {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1384: function gitHubHeaders(accept = "application/vnd.github+json") {
1385:   return {
1386:     "Accept": accept,
1387:     "User-Agent": "OMB-Portfolio-Builder-AI"
1388:   };
```

```
1389: }
```

parseGitHubSourceUrl - lines 1391-1418

Item	Explanation
Source location	Lines 1391-1418 (28 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function parseGitHubSourceUrl(url) {
Touches	Mostly local calculations or local UI state.
Calls detected	URL, toLowerCase, split, filter, slice, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 7 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1391: function parseGitHubSourceUrl(url) {
1392:   try {
1393:     const parsed = new URL(url);
1394:     const host = parsed.hostname.toLowerCase();
1395:     const parts = parsed.pathname.split("/").filter(Boolean);
1396:     if (host === "raw.githubusercontent.com" && parts.length >= 4) {
1397:       return {
1398:         type: "file",
1399:         owner: parts[0],
1400:         repo: parts[1],
1401:         branch: parts[2],
1402:         filePath: parts.slice(3).join("/")
1403:       };
1404:     }
1405:     if (host !== "github.com" || !parts.length) return null;
1406:     if (parts.length === 1) return { type: "profile", owner: parts[0] };
1407:     const base = { owner: parts[0], repo: parts[1] };
1408:     if (parts[2] === "blob" && parts.length >= 5) {
1409:       return { ...base, type: "file", branch: parts[3], filePath: parts.slice(4).join("/") };
1410:     }
1411:     if (parts[2] === "tree" && parts.length >= 4) {
1412:       return { ...base, type: "repo", branch: parts[3] };
1413:     }
1414:     return { ...base, type: "repo" };
1415:   } catch {
1416:     return null;
1417:   }
1418: }
```

fetchGitHubJson - lines 1420-1424

Item	Explanation
Source location	Lines 1420-1424 (5 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchGitHubJson(url) {
Touches	network/API requests
Calls detected	fetch, gitHubHeaders, json
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1420: async function fetchGitHubJson(url) {
1421:   const response = await fetch(url, { headers: gitHubHeaders() });
1422:   if (!response.ok) return null;
```

```
1423:   return response.json().catch(() => null);
1424: }
```

fetchLimitedText - lines 1426-1434

Item	Explanation
Source location	Lines 1426-1434 (9 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchLimitedText(url, maxLength = 12000) {
Touches	network/API requests
Calls detected	fetch, gitHubHeaders, clampText, text, get, stripHtmlToText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1426: async function fetchLimitedText(url, maxLength = 12000) {
1427:   const response = await fetch(url, {
1428:     headers: gitHubHeaders("text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1")
1429:   });
1430:   if (!response.ok) return "";
1431:   const rawText = clampText(await response.text(), maxLength);
1432:   const contentType = response.headers.get("Content-Type") || "";
1433:   return /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1434: }
```

githubQuestionTokens - lines 1436-1442

Item	Explanation
Source location	Lines 1436-1442 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function githubQuestionTokens(question = "") {
Touches	authentication/security data
Calls detected	Set, toLowerCase, replace, split, filter, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1436: function githubQuestionTokens(question = "") {
1437:   return [...new Set(String(question || "")
1438:     .toLowerCase()
1439:     .replace(/[^a-z0-9_#+.\s-]+/g, " ")
1440:     .split(/\s+/)
1441:     .filter((token) => token.length >= 3 && !["the", "and", "with", "from", "show", "code", "file",
"github"].includes(token)))];
1442: }
```

scoreGitHubFile - lines 1444-1455

Item	Explanation
Source location	Lines 1444-1455 (12 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function scoreGitHubFile(filePath = "", question = "") {

Item	Explanation
Touches	filesystem, authentication/security data
Calls detected	toLowerCase, githubQuestionTokens, b, forEach, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1444: function scoreGitHubFile(filePath = "", question = "") {
1445:   const cleanPath = filePath.toLowerCase();
1446:   const tokens = githubQuestionTokens(question);
1447:   let score = githubTextFilePattern.test(cleanPath) ? 10 : 0;
1448:   if (/readme\.md$/.test(cleanPath)) score += 55;
1449:   if (/\.(c|h|cpp|hpp|py|js|mjs|ts|v|sv|vhdl?|ino|tcl|xdc|sdc)$/i.test(cleanPath)) score += 28;
1450:   if (/\b(test|tb|bench|sim|simulation|src|firmware|hardware|rtl|driver|main)\b/i.test(cleanPath)) score
+= 12;
1451:   tokens.forEach((token) => {
1452:     if (cleanPath.includes(token)) score += 18;
1453:   });
1454:   return score;
1455: }
```

wantsGitHubCode - lines 1457-1459

Item	Explanation
Source location	Lines 1457-1459 (3 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function wantsGitHubCode(question = "") {
Touches	filesystem
Calls detected	b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1457: function wantsGitHubCode(question = "") {
1458:   return /\b(code|source|snippet|implementation|firmware|driver|module|verilog|vhdl|python|javascript|typ
escript|c\+\+|cpp|c\s+code|pull|show|display)\b/i.test(question);
1459: }
```

scoreGitHubRepo - lines 1461-1481

Item	Explanation
Source location	Lines 1461-1481 (21 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function scoreGitHubRepo(repo = {}, question = "") {
Touches	filesystem, authentication/security data
Calls detected	githubQuestionTokens, isArray, join, toLowerCase, forEach, includes, b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1461: function scoreGitHubRepo(repo = {}, question = "") {
 1462:   const tokens = githubQuestionTokens(question);
 1463:   const haystack = [
 1464:     repo.full_name,
 1465:     repo.name,
 1466:     repo.description,
 1467:     repo.language,
 1468:     ...(Array.isArray(repo.topics) ? repo.topics : [])
 1469:   ].join(" ").toLowerCase();
 1470:   let score = repo.fork ? -10 : 12;
 1471:   if (!repo.archived) score += 6;
 1472:   if (repo.language) score += 4;
 1473:   if (repo.name && !/\.github\.io$/i.test(repo.name)) score += 3;
 1474:   tokens.forEach((token) => {
 1475:     if (haystack.includes(token)) score += 22;
 1476:   });
 1477:   if (/\b(vco|oscillator|pwm|analog|mixed|pcb|firmware|stm32|fpga|asic|verilog|embedded|signal|design)\b/
i.test(haystack)) {
 1478:     score += 12;
 1479:   }
 1480:   return score;
 1481: }
```

fetchGitHubProfileSource - lines 1483-1543

Item	Explanation
Source location	Lines 1483-1543 (61 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchGitHubProfileSource(source = {}, parsed = {}) {
Touches	network/API requests
Calls detected	all, fetchGitHubJson, isArray, wantsGitHubCode, sort, scoreGitHubRepo, slice, fetchGitHubRepositorySource, trim, push, map, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
 1483: async function fetchGitHubProfileSource(source = {}, parsed = {}) {
 1484:   const owner = parsed.owner;
 1485:   const [profile, repos] = await Promise.all([
 1486:     fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}`),
 1487:
fetchGitHubJson(`https://api.github.com/users/${encodeURIComponent(owner)}/repos?per_page=30&sort=updated`)
 1488:   ]);
 1489:   const repoList = Array.isArray(repos) ? repos : [];
 1490:   const includeCode = wantsGitHubCode(source.question || "");
 1491:   const selectedRepos = includeCode
 1492:     ? [...repoList]
 1493:       .sort((a, b) => scoreGitHubRepo(b, source.question || "") - scoreGitHubRepo(a, source.question ||
"""))
 1494:       .slice(0, 3)
 1495:     : [];
 1496:   const repoBlocks = [];
 1497:
 1498:   for (const repo of selectedRepos) {
 1499:     if (!repo?.name || !repo?.html_url) continue;
 1500:     const repoSource = await fetchGitHubRepositorySource({
 1501:       ...source,
 1502:       context: `Public GitHub repository selected from ${owner}'s profile`,
 1503:       label: repo.full_name,
 1504:       maxCodeFiles: 3,
 1505:       maxFileTextLength: 3600,
 1506:       maxRepositoryTextLength: 9000,
 1507:       url: repo.html_url
 1508:     }, {
 1509:       owner,
 1510:       repo: repo.name,
```

```
      type: "repo",
      branch: repo.default_branch
    });
    if (repoSource?.text?.trim()) repoBlocks.push(repoSource.text.trim());
  }

  const lines = [
    `GitHub public profile: ${owner}`,
    profile?.name ? `Name: ${profile.name}` : "",
    profile?.bio ? `Bio: ${profile.bio}` : "",
    profile?.html_url ? `Profile URL: ${profile.html_url}` : source.url,
    "",
    "Public repositories visible from the profile:",
    ...repoList.slice(0, 20).map((repo) => [
      `- ${repo.full_name}`,
      repo.description ? `: ${repo.description}` : "",
      repo.language ? ` | Language: ${repo.language}` : "",
      repo.html_url ? ` | URL: ${repo.html_url}` : "",
      repo.updated_at ? ` | Updated: ${repo.updated_at}` : ""
    ].join("")),
    includeCode && selectedRepos.length ? "" : "",
    includeCode && selectedRepos.length ? "Selected public repositories inspected for source code:" : "",
    ...selectedRepos.map((repo) => `- ${repo.full_name}${repo.language ? ` | Language: ${repo.language}`
: ""}${repo.description ? ` | ${repo.description}` : ""}`),
    ...repoBlocks.flatMap((block) => ["", "---", block])
  ].filter(Boolean);
  return {
    context: source.context || "Public GitHub profile",
    title: source.title || source.label || `GitHub profile: ${owner}`,
    type: includeCode ? "public GitHub profile and selected repository code" : "public GitHub profile",
    url: source.url,
    text: clampText(lines.join("\n"), includeCode ? 30000 : 9000)
  };
}
```

fetchGitHubRepositorySource - lines 1545-1612

Item	Explanation
Source location	Lines 1545-1612 (68 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
Touches	network/API requests
Calls detected	fetchGitHubJson, wantsGitHubCode, max, min, isFinite, filter, fetchLimitedText, trim, push, clampText, join, isArray
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	5 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
async function fetchGitHubRepositorySource(source = {}, parsed = {}) {
  const { owner, repo } = parsed;
  const metadata = await
fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}`);
  const branch = parsed.branch || metadata?.default_branch || "main";
  const question = source.question || "";
  const requestedMaxCodeFiles = Number(source.maxCodeFiles || (wantsGitHubCode(question) ? 6 : 3));
  const maxCodeFiles = Math.max(1, Math.min(8, Number.isFinite(requestedMaxCodeFiles) ?
requestedMaxCodeFiles : 3));
  const requestedFileTextLength = Number(source.maxFileTextLength || (wantsGitHubCode(question) ? 7000 :
3000));
  const maxFileTextLength = Math.max(1000, Math.min(10000, Number.isFinite(requestedFileTextLength) ?
requestedFileTextLength : 3000));
  const requestedRepoTextLength = Number(source.maxRepositoryTextLength || (wantsGitHubCode(question) ?
22000 : 14000));
  const maxRepositoryTextLength = Math.max(5000, Math.min(26000, Number.isFinite(requestedRepoTextLength)
  ? requestedRepoTextLength : 14000));
  const lines = [
    `GitHub repository: ${owner}/${repo}`,
    metadata?.description ? `Description: ${metadata.description}` : "",
    metadata?.language ? `Primary language: ${metadata.language}` : "",
    metadata?.html_url ? `Repository URL: ${metadata.html_url}` : source.url,
    metadata?.default_branch ? `Default branch: ${metadata.default_branch}` : "",
    ""
  ].filter(Boolean);

  if (parsed.type === "file" && parsed.filePath) {
    const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${parsed.filePath}`;
```

```
    const text = await fetchLimitedText(rawUrl, 14000);
    if (text.trim()) lines.push(`Source file: ${parsed.filePath}`, text.trim());
    return {
      context: source.context || "Public GitHub source file",
      title: source.title || source.label || `${owner}/${repo}/${parsed.filePath}`,
      type: "public GitHub source file",
      url: source.url,
      text: clampText(lines.join("\n"), 14000)
    };
  }

  const tree = await fetchGitHubJson(`https://api.github.com/repos/${encodeURIComponent(owner)}/${encodeURIComponent(repo)}/git/trees/${encodeURIComponent(branch)}?recursive=1`);
  const files = Array.isArray(tree?.tree)
    ? tree.tree
      .filter((item) => item.type === "blob" && item.path && githubTextFilePattern.test(item.path) && !githubSkipFilePattern.test(item.path))
      .sort((a, b) => scoreGitHubFile(b.path, question) - scoreGitHubFile(a.path, question))
    : [];

  const readmePath = files.find((file) => /(^|\/)readme\.md$/i.test(file.path))?.path || "README.md";
  const readmeText = await fetchLimitedText(`https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${readmePath}`, 9000).catch(() => "");
  if (readmeText.trim()) lines.push(`README excerpt from ${readmePath}:`, readmeText.trim(), "");

  if (files.length) {
    lines.push("Repository text/code file list:", ...files.slice(0, 30).map((file) => `- ${file.path}`), "");
  }

  const includeCode = wantsGitHubCode(question);
  const selectedFiles = files
    .filter((file) => !/readme\.md$/i.test(file.path))
    .slice(0, maxCodeFiles);

  for (const file of selectedFiles) {
    const rawUrl = `https://raw.githubusercontent.com/${owner}/${repo}/${branch}/${file.path}`;
    const fileText = await fetchLimitedText(rawUrl, maxFileTextLength).catch(() => "");
    if (!fileText.trim()) continue;
    lines.push(`Source file: ${file.path}`, fileText.trim(), "");
  }

  return {
    context: source.context || "Public GitHub repository",
    title: source.title || source.label || `${owner}/${repo}`,
    type: includeCode ? "public GitHub repository and code" : "public GitHub repository",
    url: source.url,
    text: clampText(lines.join("\n"), maxRepositoryTextLength)
  };
}
```

fetchGitHubSourceText - lines 1614-1619

Item	Explanation
Source location	Lines 1614-1619 (6 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchGitHubSourceText(source = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	parseGitHubSourceUrl, fetchGitHubProfileSource, fetchGitHubRepositorySource
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
async function fetchGitHubSourceText(source = {}) {
  const parsed = parseGitHubSourceUrl(source.url || "");
  if (!parsed) return null;
  if (parsed.type === "profile") return fetchGitHubProfileSource(source, parsed);
  return fetchGitHubRepositorySource(source, parsed);
}
```

readLocalSourceText - lines 1621-1639

Item	Explanation
Source location	Lines 1621-1639 (19 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function readLocalSourceText(url) {
Touches	filesystem
Calls detected	URL, Set, has, toLowerCase, replace, includes, extname, readFile, resolveInsideRoot
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1621: async function readLocalSourceText(url) {
1622:   let parsed;
1623:   try {
1624:     parsed = new URL(url);
1625:   } catch {
1626:     return "";
1627:   }
1628:   const localHosts = new Set(["localhost", "127.0.0.1"]);
1629:   if (!localHosts.has(parsed.hostname.toLowerCase())) return "";
1630:   const pathname = decodeURIComponent(parsed.pathname.replace(/^\/+/, ""));
1631:   if (!pathname || pathname.includes("..")) return "";
1632:   const ext = path.extname(pathname).toLowerCase();
1633:   if (![".txt", ".md", ".json", ".csv", ".log", ".xml", ".js", ".mjs", ".css", ".c", ".h", ".cpp",
".hpp", ".py", ".v", ".sv", ".spice", ".cir", ".net", ".asc", ".sch", ".kicad_sch", ".kicad_pcb"].includes(ext))
return "";
1634:   try {
1635:     return await readFile(resolveInsideRoot(pathname), "utf8");
1636:   } catch {
1637:     return "";
1638:   }
1639: }
```

fetchSourceText - lines 1641-1679

Item	Explanation
Source location	Lines 1641-1679 (39 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function fetchSourceText(source = {}) {
Touches	network/API requests
Calls detected	sourceLooksTextual, sourceUrlAllowed, fetchGitHubSourceText, trim, readLocalSourceText, clampText, replace, fetch, get, text, stripHtmlToText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 8 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1641: async function fetchSourceText(source = {}) {
1642:   const url = String(source.url || "");
1643:   if (!url || !sourceLooksTextual(source) || !sourceUrlAllowed(url)) return null;
1644:   if (/github\.com|raw\.githubusercontent\.com/i.test(url)) {
1645:     const githubText = await fetchGitHubSourceText(source);
1646:     if (githubText?.text?.trim()) return githubText;
1647:   }
1648:   const localText = await readLocalSourceText(url);
1649:   if (localText.trim()) {
1650:     return {
1651:       context: source.context || "",
1652:       title: source.title || source.label || url,
1653:       type: source.type || source.kind || "text source",
1654:       url,
```

```
1655:       text: clampText(localText.replace(/\s+\n/g, "\n").trim(), 9000)
1656:     };
1657:   }
1658:
1659:   try {
1660:     const response = await fetch(url, {
1661:       headers: { "Accept": "text/plain,text/markdown,text/html,application/json;q=0.8,*/*;q=0.1" }
1662:     });
1663:     if (!response.ok) return null;
1664:     const contentType = response.headers.get("Content-Type") || "";
1665:     if (!/text|json|xml|html|markdown/i.test(contentType)) return null;
1666:     const rawText = clampText(await response.text(), 12000);
1667:     const text = /html/i.test(contentType) ? stripHtmlToText(rawText) : rawText;
1668:     if (!text.trim()) return null;
1669:     return {
1670:       context: source.context || "",
1671:       title: source.title || source.label || url,
1672:       type: source.type || source.kind || "public source",
1673:       url,
1674:       text: clampText(text.trim(), 9000)
1675:     };
1676:   } catch {
1677:     return null;
1678:   }
1679: }
```

enrichPortfolioContext - lines 1681-1692

Item	Explanation
Source location	Lines 1681-1692 (12 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function enrichPortfolioContext(context = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	isArray, slice, map, all, filter
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1681: async function enrichPortfolioContext(context = {}) {
1682:   const question = String(context.question || "");
1683:   const sources = Array.isArray(context.sourceFetches)
1684:     ? context.sourceFetches.slice(0, 10).map((source) => ({ ...source, question: source.question ||
question }))
1685:     : [];
1686:   const sourceExcerpts = (await Promise.all(sources.map(fetchSourceText))).filter(Boolean);
1687:   return {
1688:     ...context,
1689:     sourceExcerpts,
1690:     sourceFetchPolicy: "Fetched excerpts are limited to safe text-like same-site, GitHub/raw GitHub,
LinkedIn, and public portfolio URLs. GitHub repository links can be expanded into repository metadata, README
text, and selected public source files when a question asks for code. PDFs/images are represented by captions,
metadata, filenames, and companion text files when available."
1691:   };
1692: }
```

cleanConversationHistory - lines 1694-1703

Item	Explanation
Source location	Lines 1694-1703 (10 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function cleanConversationHistory(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	isArray, slice, map, clampText, trim, filter
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1694: function cleanConversationHistory(value) {
1695:   if (!Array.isArray(value)) return [];
1696:   return value
1697:     .slice(-8)
1698:     .map((item) => ({
1699:       role: item?.role === "assistant" ? "assistant" : "user",
1700:       content: clampText(item?.content, 1400).trim()
1701:     }))
1702:     .filter((item) => item.content);
1703: }
```

extractOpenAiText - lines 1705-1717

Item	Explanation
Source location	Lines 1705-1717 (13 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function extractOpenAiText(data) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, isArray, forEach, push, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1705: function extractOpenAiText(data) {
1706:   if (typeof data?.output_text === "string" && data.output_text.trim()) return data.output_text.trim();
1707:   const output = Array.isArray(data?.output) ? data.output : [];
1708:   const chunks = [];
1709:
1710:   output.forEach((item) => {
1711:     (item.content || []).forEach((content) => {
1712:       if (typeof content.text === "string" && content.text.trim()) chunks.push(content.text.trim());
1713:     });
1714:   });
1715:
1716:   return chunks.join("\n\n").trim();
1717: }
```

extractOllamaText - lines 1719-1723

Item	Explanation
Source location	Lines 1719-1723 (5 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function extractOllamaText(data = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1719: function extractOllamaText(data = {}) {
1720:   if (typeof data.message?.content === "string") return data.message.content.trim();
1721:   if (typeof data.response === "string") return data.response.trim();
1722:   return "";
1723: }
```

callOllamaPortfolioAi - lines 1725-1773

Item	Explanation
Source location	Lines 1725-1773 (49 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
Touches	network/API requests
Calls detected	AbortController, abort, fetch, replace, stringify, clampText, join, json, extractOllamaText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 4 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1725: async function callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch }) {
1726:   const host = process.env.OLLAMA_HOST || "http://127.0.0.1:11434";
1727:   const model = process.env.OLLAMA_MODEL || "llama3.2";
1728:   const controller = new AbortController();
1729:   const timeout = setTimeout(() => controller.abort(), Number(process.env.OLLAMA_TIMEOUT_MS || 45000));
1730:   try {
1731:     const response = await fetch(`${host.replace(/\/+$/, "")}/api/chat`, {
1732:       method: "POST",
1733:       headers: { "Content-Type": "application/json" },
1734:       signal: controller.signal,
1735:       body: JSON.stringify({
1736:         model,
1737:         stream: false,
1738:         messages: [
1739:           { role: "system", content: portfolioAiInstructions },
1740:           {
1741:             role: "user",
1742:             content: [
1743:               `Visitor question: ${question}`,
1744:               `Question intent: ${intent}`,
1745:               `Web/public lookup requested by browser: ${allowWebSearch ? "yes" : "no"}`,
1746:               "",
1747:               "Recent conversation JSON:",
1748:               clampText(JSON.stringify(conversation, null, 2), 8000),
1749:               "",
1750:               "Portfolio context JSON:",
1751:               clampText(JSON.stringify(context, null, 2), 18000)
1752:             ].join("\n")
1753:           }
1754:         ],
1755:         options: {
1756:           temperature: Number(process.env.OLLAMA_TEMPERATURE || 0.35)
1757:         }
1758:       })
1759:     });
1760:     const data = await response.json().catch(() => ({}));
1761:     if (!response.ok) {
1762:       return { ok: false, error: data?.error || `Ollama returned HTTP ${response.status}.` };
1763:     }
1764:     const answer = extractOllamaText(data);
1765:     return answer
1766:       ? { ok: true, answer, model }
1767:       : { ok: false, error: "Ollama did not return an answer." };
1768:   } catch (error) {
1769:     return { ok: false, error: error?.name === "AbortError" ? "Ollama timed out." : "Ollama is not
reachable on this machine." };
1770:   } finally {
1771:     clearTimeout(timeout);
```

```
1772:   }
1773: }
```

ruleBasedConversationAnswer - lines 1775-1786

Item	Explanation
Source location	Lines 1775-1786 (12 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function ruleBasedConversationAnswer(question = "") {
Touches	filesystem
Calls detected	toLowerCase, b
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1775: function ruleBasedConversationAnswer(question = "") {
1776:   const clean = String(question || "").toLowerCase();
1777:   if (/\b(thanks|thank you|appreciate it)\b/.test(clean)) {
1778:     return "You're welcome. I can keep helping with this portfolio, project evidence, resume links, or
related electronics and embedded-systems questions.";
1779:   }
1780:
1781:   if (/\b(who are you|what are you)\b/.test(clean)) {
1782:     return "I am this portfolio's assistant. I can help visitors explore the saved engineering work,
explain project details, summarize portfolio evidence, and answer related electronics, embedded, analog,
digital, FPGA, ASIC, PCB, and firmware questions.";
1783:   }
1784:
1785:   return "Hi. I can help you explore this portfolio, explain projects, summarize project evidence, open
relevant sections, or answer related electronics and embedded-systems questions. You can ask about a specific
project, a tool like KiCad or STM32CubeIDE, or a general topic such as embedded systems, op amps, FPGA design,
ASICs, or PCB testing.";
1786: }
```

isLocalRequest - lines 1788-1791

Item	Explanation
Source location	Lines 1788-1791 (4 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function isLocalRequest(request) {
Touches	Mostly local calculations or local UI state.
Calls detected	No obvious named calls detected by the generator.
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1788: function isLocalRequest(request) {
1789:   const address = request.socket.remoteAddress || "";
1790:   return address === "127.0.0.1" || address === "::1" || address === "::ffff:127.0.0.1";
1791: }
```

readJsonFile - lines 1793-1795

Item	Explanation
Source location	Lines 1793-1795 (3 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function readJsonFile(filePath) {
Touches	filesystem
Calls detected	parse, readFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1793: async function readJsonFile(filePath) {
1794:   return JSON.parse(await readFile(filePath, "utf8"));
1795: }
```

readRequestJson - lines 1797-1810

Item	Explanation
Source location	Lines 1797-1810 (14 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function readRequestJson(request) {
Touches	Mostly local calculations or local UI state.
Calls detected	Error, push, parse, concat, toString, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1797: async function readRequestJson(request) {
1798:   const chunks = [];
1799:   let size = 0;
1800:
1801:   for await (const chunk of request) {
1802:     size += chunk.length;
1803:     if (size > 60 * 1024 * 1024) {
1804:       throw new Error("Request body is too large.");
1805:     }
1806:     chunks.push(chunk);
1807:   }
1808:
1809:   return JSON.parse((Buffer.concat(chunks).toString("utf8") || "{}").replace(/^\uFEFF/, ""));
1810: }
```

safeSegment - lines 1812-1819

Item	Explanation
Source location	Lines 1812-1819 (8 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function safeSegment(value, fallback = "item") {
Touches	Mostly local calculations or local UI state.
Calls detected	toLowerCase, trim, replace
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1812: function safeSegment(value, fallback = "item") {
1813:   const segment = String(value || fallback)
1814:     .toLowerCase()
1815:     .trim()
1816:     .replace(/[^a-z0-9._-]+/g, "-")
1817:     .replace(/^-+|-+$/g, "");
1818:   return segment || fallback;
1819: }
```

safeFileName - lines 1821-1826

Item	Explanation
Source location	Lines 1821-1826 (6 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function safeFileName(value) {
Touches	Mostly local calculations or local UI state.
Calls detected	parse, safeSegment, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1821: function safeFileName(value) {
1822:   const parsed = path.parse(String(value || "file"));
1823:   const name = safeSegment(parsed.name, "file");
1824:   const ext = safeSegment(parsed.ext.replace(".", ""), "");
1825:   return ext ? `${name}.${ext}` : name;
1826: }
```

resolveInsideRoot - lines 1828-1834

Item	Explanation
Source location	Lines 1828-1834 (7 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function resolveInsideRoot(...segments) {
Touches	filesystem
Calls detected	normalize, join, startsWith, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1828: function resolveInsideRoot(...segments) {
1829:   const target = path.normalize(path.join(root, ...segments));
1830:   if (!target.startsWith(root)) {
1831:     throw new Error("Resolved path is outside the workspace.");
1832:   }
```

```
1833:   return target;
1834: }
```

resolveInsidePortfolioRoot - lines 1836-1843

Item	Explanation
Source location	Lines 1836-1843 (8 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function resolveInsidePortfolioRoot(...segments) {
Touches	Mostly local calculations or local UI state.
Calls detected	resolve, relative, startsWith, isAbsolute, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1836: function resolveInsidePortfolioRoot(...segments) {
1837:   const target = path.resolve(portfolioRoot, ...segments);
1838:   const relative = path.relative(portfolioRoot, target);
1839:   if (relative.startsWith("..") || path.isAbsolute(relative)) {
1840:     throw new Error("Resolved path is outside the portfolio workspace.");
1841:   }
1842:   return target;
1843: }
```

resolveInsideCompileRoot - lines 1845-1852

Item	Explanation
Source location	Lines 1845-1852 (8 source lines).
Purpose	Part of the Compile Code language/tool/build/run flow.
Declaration	function resolveInsideCompileRoot(...segments) {
Touches	Mostly local calculations or local UI state.
Calls detected	resolve, relative, startsWith, isAbsolute, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1845: function resolveInsideCompileRoot(...segments) {
1846:   const target = path.resolve(compileRoot, ...segments);
1847:   const relative = path.relative(compileRoot, target);
1848:   if (relative.startsWith("..") || path.isAbsolute(relative)) {
1849:     throw new Error("Resolved path is outside the compile workspace.");
1850:   }
1851:   return target;
1852: }
```

samePath - lines 1854-1856

Item	Explanation
Source location	Lines 1854-1856 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function samePath(left, right) {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	resolve, toLowerCase
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1854: function samePath(left, right) {
1855:   return path.resolve(left).toLowerCase() === path.resolve(right).toLowerCase();
1856: }
```

pathExists - lines 1858-1865

Item	Explanation
Source location	Lines 1858-1865 (8 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function pathExists(filePath) {
Touches	Mostly local calculations or local UI state.
Calls detected	fsAccess
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1858: async function pathExists(filePath) {
1859:   try {
1860:     await fsAccess(filePath);
1861:     return true;
1862:   } catch {
1863:     return false;
1864:   }
1865: }
```

publishAccessError - lines 1867-1877

Item	Explanation
Source location	Lines 1867-1877 (11 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function publishAccessError(message, details = "", extra = {}) {
Touches	authentication/security data
Calls detected	Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1867: function publishAccessError(message, details = "", extra = {}) {
1868:   const error = new Error(message);
```

```
1869:   error.code = "PUBLISH_AUTHORIZATION_REQUIRED";
1870:   error.details = details || publishAuthorizationHelp;
1871:   error.publishAccess = {
1872:     authorizationRequired: true,
1873:     help: publishAuthorizationHelp,
1874:     ...extra
1875:   };
1876:   return error;
1877: }
```

gitFailureText - lines 1879-1885

Item	Explanation
Source location	Lines 1879-1885 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function gitFailureText(error) {
Touches	Mostly local calculations or local UI state.
Calls detected	filter, join, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1879: function gitFailureText(error) {
1880:   return [
1881:     error?.stderr,
1882:     error?.stdout,
1883:     error?.message
1884:   ].filter(Boolean).join("\n").trim();
1885: }
```

remoteUrlForDisplay - lines 1887-1891

Item	Explanation
Source location	Lines 1887-1891 (5 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function remoteUrlForDisplay(remoteUrl = "") {
Touches	network/API requests
Calls detected	replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1887: function remoteUrlForDisplay(remoteUrl = "") {
1888:   return String(remoteUrl || "")
1889:     .replace(/^https:\/\/([^:@/]+):[^@/]+@/i, "https://$1:***@")
1890:     .replace(/^https:\/\/[^@/]+@/i, "https://");
1891: }
```

parseGitHubRemote - lines 1893-1907

Item	Explanation
Source location	Lines 1893-1907 (15 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.

Item	Explanation
Declaration	function parseGitHubRemote(remoteUrl = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, match, replace, URL, split
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1893: function parseGitHubRemote(remoteUrl = "") {
1894:   const trimmed = String(remoteUrl || "").trim();
1895:   const sshMatch = trimmed.match(/^git@github\.com:([^/]+)\/(.+?)(?:\.git)?$/i);
1896:   if (sshMatch) return { owner: sshMatch[1], repo: sshMatch[2].replace(/\.git$/i, "") };
1897:
1898:   try {
1899:     const parsed = new URL(trimmed.replace(/^git\+/, ""));
1900:     if (!/github\.com$/i.test(parsed.hostname)) return null;
1901:     const parts = parsed.pathname.replace(/^\/+/, "").split("/");
1902:     if (parts.length < 2) return null;
1903:     return { owner: parts[0], repo: parts[1].replace(/\.git$/i, "") };
1904:   } catch {
1905:     return null;
1906:   }
1907: }
```

validatePublishRemoteUrl - lines 1909-1927

Item	Explanation
Source location	Lines 1909-1927 (19 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function validatePublishRemoteUrl(value = "") {
Touches	network/API requests
Calls detected	trim, URL, toLowerCase, Error, replace, split, includes
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1909: function validatePublishRemoteUrl(value = "") {
1910:   const remoteUrl = String(value || "").trim();
1911:   if (!remoteUrl) return "";
1912:   if (/^git@github\.com:[A-Za-z0-9_.-]+\/[A-Za-z0-9_.-]+(?:\.git)?$/i.test(remoteUrl)) return remoteUrl;
1913:   try {
1914:     const parsed = new URL(remoteUrl);
1915:     if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
1916:       throw new Error("Only GitHub HTTPS repository URLs are supported by the guided setup.");
1917:     }
1918:     const parts = parsed.pathname.replace(/^\/+/, "").split("/");
1919:     if (parts.length < 2 || !parts[0] || !parts[1]) {
1920:       throw new Error("GitHub repository URL must include owner and repository name.");
1921:     }
1922:     return `https://github.com/${parts[0]}/${parts[1].replace(/\.git$/i, "")}.git`;
1923:   } catch (error) {
1924:     if (error.message?.includes("Only GitHub") || error.message?.includes("owner")) throw error;
1925:     throw new Error("Enter a GitHub repository URL such as
https://github.com/USERNAME/USERNAME.github.io.git.");
1926:   }
1927: }
```

validateCustomDomain - lines 1929-1936

Item	Explanation
Source location	Lines 1929-1936 (8 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function validateCustomDomain(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	trim, toLowerCase, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1929: function validateCustomDomain(value = "") {
1930:   const domain = String(value || "").trim().toLowerCase();
1931:   if (!domain) return "";
1932:   if (!/^(?:[a-z0-9](?:[a-z0-9-]{0,61}[a-z0-9])?\.)+[a-z]{2,63}$/i.test(domain)) {
1933:     throw new Error("Custom domain must look like example.com or portfolio.example.com.");
1934:   }
1935:   return domain;
1936: }
```

compareVersions - lines 1938-1947

Item	Explanation
Source location	Lines 1938-1947 (10 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function compareVersions(left = "", right = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	split, map, max
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1938: function compareVersions(left = "", right = "") {
1939:   const leftParts = String(left || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
1940:   const rightParts = String(right || "0").split(/[.-]/).map((part) => Number.parseInt(part, 10) || 0);
1941:   const length = Math.max(leftParts.length, rightParts.length);
1942:   for (let index = 0; index < length; index += 1) {
1943:     const delta = (leftParts[index] || 0) - (rightParts[index] || 0);
1944:     if (delta !== 0) return delta;
1945:   }
1946:   return 0;
1947: }
```

bumpPublishedAssetVersions - lines 1949-1965

Item	Explanation
Source location	Lines 1949-1965 (17 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) {
Touches	filesystem
Calls detected	join, readFile, now, toString, replace, writeFile

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1949: async function bumpPublishedAssetVersions(baseRoot = portfolioRoot) {
1950:   let html = "";
1951:   const indexPath = path.join(baseRoot, "index.html");
1952:   try {
1953:     html = await readFile(indexPath, "utf8");
1954:   } catch {
1955:     return false;
1956:   }
1957:   const version = Date.now().toString();
1958:   const next = html
1959:     .replace(/(href="styles\.css)(?:\?v=[^"]*)?"/g, `$1?v=${version}"`)
1960:     .replace(/(src="script\.js)(?:\?v=[^"]*)?"/g, `$1?v=${version}"`)
1961:     .replace(/(src="electronics-search\.js)(?:\?v=[^"]*)?"/g, `$1?v=${version}"`);
1962:   if (next === html) return false;
1963:   await writeFile(indexPath, next, "utf8");
1964:   return true;
1965: }
```

workspaceHasCompatibleSiteFiles - lines 1967-1976

Item	Explanation
Source location	Lines 1967-1976 (10 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {
Touches	filesystem
Calls detected	readFile, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1967: async function workspaceHasCompatibleSiteFiles(baseRoot = portfolioRoot) {
1968:   for (const fileName of ["index.html", "styles.css", "script.js"]) {
1969:     try {
1970:       await readFile(path.join(baseRoot, fileName));
1971:     } catch {
1972:       return false;
1973:     }
1974:   }
1975:   return true;
1976: }
```

getPublishTargetInfo - lines 1978-2044

Item	Explanation
Source location	Lines 1978-2044 (67 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function getPublishTargetInfo() {
Touches	filesystem, authentication/security data
Calls detected	samePath, workspaceHasCompatibleSiteFiles, runPublishGit, trim, remoteUrlForDisplay, parseGitHubRemote, readFile, resolveInsidePortfolioRoot, readPublishAuthCache, publishAuthCacheIsFresh, publishAuthCacheExpiresAt

Item	Explanation
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	6 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
1978: async function getPublishTargetInfo() {
1979:   const info = {
1980:     workspace: root,
1981:     portfolioWorkspace: portfolioRoot,
1982:     separatedWorkspace: !samePath(root, portfolioRoot),
1983:     defaultRepository: defaultSiteRepository,
1984:     gitBacked: false,
1985:     compatible: await workspaceHasCompatibleSiteFiles(),
1986:     remote: "",
1987:     branch: "",
1988:     repository: "",
1989:     customDomain: ""
1990:   };
1991:
1992:   try {
1993:     info.gitBacked = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim() ===
"true";
1994:   } catch {
1995:     return info;
1996:   }
1997:
1998:   try {
1999:     info.remote = remoteUrlForDisplay((await runPublishGit(["remote", "get-url",
"origin"])).stdout.trim());
2000:   } catch {
2001:     info.remote = "";
2002:   }
2003:
2004:   try {
2005:     info.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2006:   } catch {
2007:     info.branch = "";
2008:   }
2009:
2010:   const parsedRemote = parseGitHubRemote(info.remote);
2011:   info.repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` : info.remote;
2012:
2013:   try {
2014:     info.customDomain = (await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8")).trim();
2015:   } catch {
2016:     info.customDomain = "";
2017:   }
2018:
2019:   const accessShape = {
2020:     branch: info.branch,
2021:     remote: info.remote,
2022:     repository: info.repository
2023:   };
2024:   const cached = await readPublishAuthCache();
2025:   if (publishAuthCacheIsFresh(cached, accessShape)) {
2026:     info.authorizationChecked = true;
2027:     info.authorizationCached = true;
2028:     info.checkedAt = cached.checkedAt;
2029:     info.expiresAt = publishAuthCacheExpiresAt(cached);
2030:     info.extendedTrust = Boolean(cached.extendedTrust);
2031:     info.successCountLastWeek = cached.successCountLastWeek || 0;
2032:     info.trustDays = cached.trustDays || (cached.extendedTrust ? 30 : 1);
2033:   } else {
2034:     info.authorizationChecked = false;
2035:     info.authorizationCached = false;
2036:     info.checkedAt = "";
2037:     info.expiresAt = "";
2038:     info.extendedTrust = false;
2039:     info.successCountLastWeek = cached?.successCountLastWeek || 0;
2040:     info.trustDays = 0;
2041:   }
2042:
2043:   return info;
2044: }
```

runGit - lines 2046-2064

Item	Explanation
Source location	Lines 2046-2064 (19 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function runGit(args, options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	execFileAsync, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2046: async function runGit(args, options = {}) {
2047:   let lastError = null;
2048:   for (const candidate of gitCandidates) {
2049:     try {
2050:       const result = await execFileAsync(candidate, args, {
2051:         cwd: options.cwd || root,
2052:         maxBuffer: options.maxBuffer || 10 * 1024 * 1024,
2053:         timeout: options.timeout || 0,
2054:         windowsHide: options.windowsHide !== false
2055:       });
2056:       return { ...result, git: candidate };
2057:     } catch (error) {
2058:       lastError = error;
2059:       if (error.code === "ENOENT") continue;
2060:       throw error;
2061:     }
2062:   }
2063:   throw lastError || new Error("Git executable was not found.");
2064: }
```

runPublishGit - lines 2066-2068

Item	Explanation
Source location	Lines 2066-2068 (3 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function runPublishGit(args, options = {}) {
Touches	Mostly local calculations or local UI state.
Calls detected	runGit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2066: async function runPublishGit(args, options = {}) {
2067:   return runGit(args, { ...options, cwd: options.cwd || portfolioRoot });
2068: }
```

runOptionalCommand - lines 2070-2087

Item	Explanation
Source location	Lines 2070-2087 (18 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function runOptionalCommand(command, args = [], options = {}) {

Item	Explanation
Touches	Mostly local calculations or local UI state.
Calls detected	execFileAsync
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2070: async function runOptionalCommand(command, args = [], options = {}) {
2071:   try {
2072:     const result = await execFileAsync(command, args, {
2073:       cwd: options.cwd || root,
2074:       maxBuffer: options.maxBuffer || 2 * 1024 * 1024,
2075:       timeout: options.timeout || 15000,
2076:       windowsHide: options.windowsHide !== false
2077:     });
2078:     return { ok: true, stdout: result.stdout || "", stderr: result.stderr || "" };
2079:   } catch (error) {
2080:     return {
2081:       ok: false,
2082:       stdout: error.stdout || "",
2083:       stderr: error.stderr || "",
2084:       error: error.message || String(error)
2085:     };
2086:   }
2087: }
```

getGitStatus - lines 2089-2136

Item	Explanation
Source location	Lines 2089-2136 (48 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function getGitStatus() {
Touches	authentication/security data
Calls detected	runGit, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2089: async function getGitStatus() {
2090:   let git = { ok: false, stdout: "", stderr: "", error: "Git for Windows was not found." };
2091:   try {
2092:     const gitResult = await runGit(["--version"]);
2093:     git = { ok: true, stdout: gitResult.stdout || "", stderr: gitResult.stderr || "" };
2094:   } catch (error) {
2095:     git = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git for Windows was not found." };
2096:   }
2097:   let credentialManager = { ok: false, version: "", error: "Git Credential Manager was not found." };
2098:   if (git.ok) {
2099:     let gcm = { ok: false, stdout: "", stderr: "", error: "Git Credential Manager was not found." };
2100:     try {
2101:       const gcmResult = await runGit(["credential-manager", "--version"]);
2102:       gcm = { ok: true, stdout: gcmResult.stdout || "", stderr: gcmResult.stderr || "" };
2103:     } catch (error) {
2104:       gcm = { ok: false, stdout: error.stdout || "", stderr: error.stderr || "", error: error.message ||
"Git Credential Manager was not found." };
2105:     }
2106:     credentialManager = {
2107:       ok: gcm.ok,
2108:       version: (gcm.stdout || gcm.stderr || "").trim(),
2109:       error: gcm.ok ? "" : (gcm.stderr || gcm.error || "Git Credential Manager was not found.")
2110:     };
2111:   }
```

```
2112:
2113:   return {
2114:     git: {
2115:       ok: git.ok,
2116:       version: (git.stdout || git.stderr || "").trim(),
2117:       error: git.ok ? "" : (git.stderr || git.error || "Git for Windows was not found.")
2118:     },
2119:     credentialManager,
2120:     nodeRuntime: {
2121:       ok: true,
2122:       version: process.versions.node,
2123:       bundled: true,
2124:       note: "Bundled inside the desktop app. No manual Node.js install is required."
2125:     },
2126:     pnpm: {
2127:       ok: true,
2128:       required: false,
2129:       note: "pnpm is only needed by developers building the installer from source."
2130:     },
2131:     app: {
2132:       version: process.env.OMB_APP_VERSION || packageJson.version || "0.0.0",
2133:       desktopApp: process.env.OMB_DESKTOP_APP === "1"
2134:     }
2135:   };
2136: }
```

publishPathIsStageable - lines 2138-2152

Item	Explanation
Source location	Lines 2138-2152 (15 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function publishPathIsStageable(relativePath) {
Touches	Mostly local calculations or local UI state.
Calls detected	fsAccess, resolveInsidePortfolioRoot, runPublishGit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2138: async function publishPathIsStageable(relativePath) {
2139:   try {
2140:     await fsAccess(resolveInsidePortfolioRoot(relativePath));
2141:     return true;
2142:   } catch {
2143:     // Missing files can still be stageable if Git already tracks them and they were deleted.
2144:   }
2145:
2146:   try {
2147:     await runPublishGit(["ls-files", "--error-unmatch", "--", relativePath]);
2148:     return true;
2149:   } catch {
2150:     return false;
2151:   }
2152: }
```

stageablePublishPaths - lines 2154-2160

Item	Explanation
Source location	Lines 2154-2160 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function stageablePublishPaths(pathsToCheck) {
Touches	Mostly local calculations or local UI state.
Calls detected	publishPathIsStageable, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2154: async function stageablePublishPaths(pathsToCheck) {
2155:   const stageable = [];
2156:   for (const relativePath of pathsToCheck) {
2157:     if (await publishPathIsStageable(relativePath)) stageable.push(relativePath);
2158:   }
2159:   return stageable;
2160: }
```

runGitWithInput - lines 2162-2204

Item	Explanation
Source location	Lines 2162-2204 (43 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function runGitWithInput(args, input, options = {}) {
Touches	external processes
Calls detected	spawn, on, toString, once, resolve, Error, reject, end
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2162: async function runGitWithInput(args, input, options = {}) {
2163:   let lastError = null;
2164:   for (const candidate of gitCandidates) {
2165:     try {
2166:       return await new Promise((resolve, reject) => {
2167:         const child = spawn(candidate, args, {
2168:           cwd: options.cwd || portfolioRoot,
2169:           windowsHide: true,
2170:           stdio: ["pipe", "pipe", "pipe"]
2171:         });
2172:         let stdout = "";
2173:         let stderr = "";
2174:
2175:         child.stdout.on("data", (chunk) => {
2176:           stdout += chunk.toString();
2177:         });
2178:         child.stderr.on("data", (chunk) => {
2179:           stderr += chunk.toString();
2180:         });
2181:         child.once("error", reject);
2182:         child.once("close", (code) => {
2183:           if (code === 0) {
2184:             resolve({ stdout, stderr, git: candidate });
2185:             return;
2186:           }
2187:           const error = new Error(stderr || stdout || `Git exited with code ${code}.`);
2188:           error.stdout = stdout;
2189:           error.stderr = stderr;
2190:           error.code = code;
2191:           error.git = candidate;
2192:           reject(error);
2193:         });
2194:
2195:         child.stdin.end(input);
2196:       });
2197:     } catch (error) {
2198:       lastError = error;
2199:       if (error.code === "ENOENT") continue;
2200:       throw error;
2201:     }
2202:   }
2203:   throw lastError || new Error("Git executable was not found.");
2204: }
```

storeGitCredentials - lines 2206-2237

Item	Explanation
Source location	Lines 2206-2237 (32 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function storeGitCredentials(remoteUrl, username, password) {
Touches	authentication/security data
Calls detected	trim, Error, URL, toLowerCase, replace, runPublishGit, join, runGitWithInput
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2206: async function storeGitCredentials(remoteUrl, username, password) {
2207:   const cleanUsername = String(username || "").trim();
2208:   const cleanPassword = String(password || "");
2209:   if (!cleanUsername && !cleanPassword) return { stored: false };
2210:   if (!cleanUsername || !cleanPassword) {
2211:     throw new Error("Both username and password/token are required when credentials are provided.");
2212:   }
2213:
2214:   let parsed = null;
2215:   try {
2216:     parsed = new URL(remoteUrl);
2217:   } catch {
2218:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2219:   }
2220:
2221:   if (parsed.protocol !== "https:" || parsed.hostname.toLowerCase() !== "github.com") {
2222:     throw new Error("Username and password/token credentials require a GitHub HTTPS repository URL.");
2223:   }
2224:
2225:   const pathName = parsed.pathname.replace(/^\/+/, "");
2226:   await runPublishGit(["config", "--local", "credential.useHttpPath", "true"]);
2227:   const credentialInput = [
2228:     "protocol=https",
2229:     "host=github.com",
2230:     `path=${pathName}`,
2231:     `username=${cleanUsername}`,
2232:     `password=${cleanPassword}`,
2233:     ""
2234:   ].join("\n");
2235:   await runGitWithInput(["credential", "approve"], credentialInput, { cwd: portfolioRoot });
2236:   return { stored: true, username: cleanUsername };
2237: }
```

validateCredentialPair - lines 2239-2246

Item	Explanation
Source location	Lines 2239-2246 (8 source lines).
Purpose	Part of authentication, authorization, or credential checking.
Declaration	function validateCredentialPair(username, password) {
Touches	authentication/security data
Calls detected	trim, Error
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2239: function validateCredentialPair(username, password) {
```

```
2240:   const cleanUsername = String(username || "").trim();
2241:   const cleanPassword = String(password || "");
2242:   if ((cleanUsername && !cleanPassword) || (!cleanUsername && cleanPassword)) {
2243:     throw new Error("Both username and password/token are required when credentials are provided.");
2244:   }
2245:   return { cleanUsername, cleanPassword };
2246: }
```

normalizeGitBranchName - lines 2248-2327

Item	Explanation
Source location	Lines 2248-2327 (80 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function normalizeGitBranchName(value = "") {
Touches	filesystem, authentication/security data
Calls detected	trim, includes, endsWith, startsWith, detectRemoteDefaultBranch, runPublishGit, match, originRemoteExists, setOriginRemote, checkoutPublishBranch, verifyPublishWriteAccess, safeSegment
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	10 await expression(s), 10 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2248: function normalizeGitBranchName(value = "") {
2249:   const branch = String(value || "").trim();
2250:   if (
2251:     !branch ||
2252:     branch.length > 180 ||
2253:     branch.includes("..") ||
2254:     branch.endsWith(".") ||
2255:     branch.includes("@{") ||
2256:     /[\\:\s~^?*\[\]\x00-\x1f]/.test(branch) ||
2257:     branch.startsWith("/") ||
2258:     branch.endsWith("/") ||
2259:     branch.includes("//")
2260:   ) {
2261:     return "";
2262:   }
2263:   return branch;
2264: }
2265:
2266: async function detectRemoteDefaultBranch(remoteUrl = "") {
2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+?)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }
2276:
2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }
2285:
2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }
2293:
2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
```

```
2302:   }
2303:   return cleanBranch;
2304: }
2305:
2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }
2324:
2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);
```

detectRemoteDefaultBranch - lines 2266-2275

Item	Explanation
Source location	Lines 2266-2275 (10 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function detectRemoteDefaultBranch(remoteUrl = "") {
Touches	filesystem
Calls detected	runPublishGit, match, normalizeGitBranchName
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2266: async function detectRemoteDefaultBranch(remoteUrl = "") {
2267:   if (!remoteUrl) return "";
2268:   try {
2269:     const result = await runPublishGit(["ls-remote", "--symref", remoteUrl, "HEAD"], { timeout: 45000 });
2270:     const match = String(result.stdout || "").match(/^ref:\s+refs\/heads\/(.+?)\s+HEAD$/m);
2271:     return normalizeGitBranchName(match?.[1] || "");
2272:   } catch {
2273:     return "";
2274:   }
2275: }
```

originRemoteExists - lines 2277-2284

Item	Explanation
Source location	Lines 2277-2284 (8 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function originRemoteExists() {
Touches	Mostly local calculations or local UI state.
Calls detected	runPublishGit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2277: async function originRemoteExists() {
2278:   try {
2279:     await runPublishGit(["remote", "get-url", "origin"]);
2280:     return true;
2281:   } catch {
2282:     return false;
2283:   }
2284: }
```

setOriginRemote - lines 2286-2292

Item	Explanation
Source location	Lines 2286-2292 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function setOriginRemote(remoteUrl) {
Touches	Mostly local calculations or local UI state.
Calls detected	originRemoteExists, runPublishGit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2286: async function setOriginRemote(remoteUrl) {
2287:   if (await originRemoteExists()) {
2288:     await runPublishGit(["remote", "set-url", "origin", remoteUrl]);
2289:   } else {
2290:     await runPublishGit(["remote", "add", "origin", remoteUrl]);
2291:   }
2292: }
```

checkoutPublishBranch - lines 2294-2304

Item	Explanation
Source location	Lines 2294-2304 (11 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function checkoutPublishBranch(branch) {
Touches	filesystem
Calls detected	normalizeGitBranchName, runPublishGit, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2294: async function checkoutPublishBranch(branch) {
2295:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2296:   const current = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2297:   if (current === cleanBranch) return cleanBranch;
2298:   try {
2299:     await runPublishGit(["checkout", cleanBranch]);
2300:   } catch {
2301:     await runPublishGit(["checkout", "-B", cleanBranch]);
```

```
2302:   }
2303:   return cleanBranch;
2304: }
```

verifyPublishWriteAccess - lines 2306-2323

Item	Explanation
Source location	Lines 2306-2323 (18 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function verifyPublishWriteAccess(access = {}) {
Touches	authentication/security data
Calls detected	safeSegment, hostname, now, runPublishGit, publishAccessError, gitFailureText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2306: async function verifyPublishWriteAccess(access = {}) {
2307:   const checkBranch = `omb-auth-check-${safeSegment(os.hostname(), "device")}-${Date.now()}`;
2308:   try {
2309:     await runPublishGit(["push", "--dry-run", "origin", `HEAD:refs/heads/${checkBranch}`], {
2310:       timeout: 60 * 1000
2311:     });
2312:     return {
2313:       method: "temporary-dry-run-ref",
2314:       ref: checkBranch
2315:     };
2316:   } catch (error) {
2317:     throw publishAccessError(
2318:       "GitHub authorization is required before this website can be changed.",
2319:       gitFailureText(error),
2320:       access
2321:     );
2322:   }
2323: }
```

ensurePublishHeadForWriteCheck - lines 2325-2342

Item	Explanation
Source location	Lines 2325-2342 (18 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function ensurePublishHeadForWriteCheck() {
Touches	authentication/security data
Calls detected	runPublishGit
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2325: async function ensurePublishHeadForWriteCheck() {
2326:   try {
2327:     await runPublishGit(["rev-parse", "--verify", "HEAD"]);
2328:     return { created: false };
2329:   } catch {
2330:     await runPublishGit([
2331:       "-c",
2332:       "user.name=OMB Portfolio Builder",
2333:       "-c",
2334:       "user.email=omb-builder@localhost",
2335:       "commit",
```

```
2336:       "--allow-empty",
2337:       "-m",
2338:       "Initialize publish authorization check"
2339:     ]);
2340:     return { created: true };
2341:   }
2342: }
```

synchronizePublishBranchFromRemote - lines 2344-2373

Item	Explanation
Source location	Lines 2344-2373 (30 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function synchronizePublishBranchFromRemote(branch, access = {}) {
Touches	filesystem
Calls detected	normalizeGitBranchName, runPublishGit, trim, checkoutPublishBranch, publishAccessError, gitFailureText
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	4 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2344: async function synchronizePublishBranchFromRemote(branch, access = {}) {
2345:   const cleanBranch = normalizeGitBranchName(branch) || "main";
2346:   try {
2347:     const remoteBranch = await runPublishGit(["ls-remote", "--heads", "origin", cleanBranch], {
2348:       timeout: 45 * 1000
2349:     });
2350:     if (!String(remoteBranch.stdout || "").trim()) {
2351:       return {
2352:         branch: cleanBranch,
2353:         remoteBranchExists: false,
2354:         synchronized: false
2355:       };
2356:     }
2357:
2358:     await runPublishGit(["fetch", "origin", cleanBranch], { timeout: 2 * 60 * 1000 });
2359:     await checkoutPublishBranch(cleanBranch);
2360:     await runPublishGit(["reset", "--hard", `origin/${cleanBranch}`], { timeout: 60 * 1000 });
2361:     return {
2362:       branch: cleanBranch,
2363:       remoteBranchExists: true,
2364:       synchronized: true
2365:     };
2366:   } catch (error) {
2367:     throw publishAccessError(
2368:       "The selected website target could not be synchronized from GitHub.",
2369:       gitFailureText(error),
2370:       { ...access, branch: cleanBranch }
2371:     );
2372:   }
2373: }
```

capturePublishTargetState - lines 2375-2403

Item	Explanation
Source location	Lines 2375-2403 (29 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function capturePublishTargetState() {
Touches	filesystem
Calls detected	runPublishGit, trim, readFile, resolveInsidePortfolioRoot
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2375: async function capturePublishTargetState() {
2376:   const snapshot = {
2377:     branch: "",
2378:     customDomainExists: false,
2379:     customDomainText: "",
2380:     remote: "",
2381:     remoteExists: false
2382:   };
2383:   try {
2384:     snapshot.remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2385:     snapshot.remoteExists = true;
2386:   } catch {
2387:     snapshot.remote = "";
2388:     snapshot.remoteExists = false;
2389:   }
2390:   try {
2391:     snapshot.branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2392:   } catch {
2393:     snapshot.branch = "";
2394:   }
2395:   try {
2396:     snapshot.customDomainText = await readFile(resolveInsidePortfolioRoot("CNAME"), "utf8");
2397:     snapshot.customDomainExists = true;
2398:   } catch {
2399:     snapshot.customDomainText = "";
2400:     snapshot.customDomainExists = false;
2401:   }
2402:   return snapshot;
2403: }
```

restorePublishTargetState - lines 2405-2429

Item	Explanation
Source location	Lines 2405-2429 (25 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function restorePublishTargetState(snapshot = {}) {
Touches	filesystem
Calls detected	setOriginRemote, originRemoteExists, runPublishGit, checkoutPublishBranch, writeFile, resolveInsidePortfolioRoot, rm
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	6 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2405: async function restorePublishTargetState(snapshot = {}) {
2406:   try {
2407:     if (snapshot.remoteExists && snapshot.remote) {
2408:       await setOriginRemote(snapshot.remote);
2409:     } else if (await originRemoteExists()) {
2410:       await runPublishGit(["remote", "remove", "origin"]);
2411:     }
2412:   } catch {
2413:     // Best-effort rollback; preserve the original error for the caller.
2414:   }
2415:   try {
2416:     if (snapshot.branch) await checkoutPublishBranch(snapshot.branch);
2417:   } catch {
2418:     // Best-effort rollback; preserve the original error for the caller.
2419:   }
2420:   try {
2421:     if (snapshot.customDomainExists) {
2422:       await writeFile(resolveInsidePortfolioRoot("CNAME"), snapshot.customDomainText, "utf8");
2423:     } else {
2424:       await rm(resolveInsidePortfolioRoot("CNAME"), { force: true });
2425:     }
2426:   } catch {
2427:     // Best-effort rollback; preserve the original error for the caller.
2428:   }
```

```
2429: }
```

writeTargetCustomDomain - lines 2431-2439

Item	Explanation
Source location	Lines 2431-2439 (9 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function writeTargetCustomDomain(domain, customDomainProvided) {
Touches	filesystem
Calls detected	resolveInsidePortfolioRoot, samePath, push, resolveInsideRoot, writeFile, rm
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2431: async function writeTargetCustomDomain(domain, customDomainProvided) {
2432:   const targetPaths = [resolveInsidePortfolioRoot("CNAME")];
2433:   if (!samePath(root, portfolioRoot)) targetPaths.push(resolveInsideRoot("CNAME"));
2434:   if (customDomainProvided && domain) {
2435:     for (const targetPath of targetPaths) await writeFile(targetPath, `${domain}\n`, "utf8");
2436:   } else if (customDomainProvided && !domain) {
2437:     for (const targetPath of targetPaths) await rm(targetPath, { force: true });
2438:   }
2439: }
```

ensureGitRepository - lines 2441-2454

Item	Explanation
Source location	Lines 2441-2454 (14 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function ensureGitRepository() {
Touches	filesystem
Calls detected	mkdir, runPublishGit, trim
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	5 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2441: async function ensureGitRepository() {
2442:   await mkdir(portfolioRoot, { recursive: true });
2443:   try {
2444:     const insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
2445:     if (insideWorkTree === "true") return;
2446:   } catch {
2447:     await runPublishGit(["init"]);
2448:   }
2449:
2450:   const branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
2451:   if (!branch) {
2452:     await runPublishGit(["checkout", "-B", "main"]);
2453:   }
2454: }
```

configurePublishTarget - lines 2456-2486

Item	Explanation
Source location	Lines 2456-2486 (31 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function configurePublishTarget(options = {}) {
Touches	authentication/security data
Calls detected	call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, ensureGitRepository, setOriginRemote, detectRemoteDefaultBranch, checkoutPublishBranch, writeTargetCustomDomain, getPublishTargetInfo, storeGitCredentials
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	8 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2456: async function configurePublishTarget(options = {}) {
2457:   const {
2458:     repositoryUrl = "",
2459:     customDomain = "",
2460:     authUsername = "",
2461:     authPassword = ""
2462:   } = options;
2463:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2464:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2465:   const domain = validateCustomDomain(customDomain);
2466:   validateCredentialPair(authUsername, authPassword);
2467:
2468:   await ensureGitRepository();
2469:
2470:   if (remoteUrl) {
2471:     await setOriginRemote(remoteUrl);
2472:     const defaultBranch = await detectRemoteDefaultBranch(remoteUrl);
2473:     if (defaultBranch) await checkoutPublishBranch(defaultBranch);
2474:   }
2475:
2476:   await writeTargetCustomDomain(domain, customDomainProvided);
2477:
2478:   const currentRemote = remoteUrl || (await getPublishTargetInfo()).remote;
2479:   const credentials = await storeGitCredentials(currentRemote, authUsername, authPassword);
2480:   const target = await getPublishTargetInfo();
2481:   return {
2482:     ...target,
2483:     credentialsStored: credentials.stored,
2484:     credentialUsername: credentials.username || ""
2485:   };
2486: }
```

readPublishAuthCache - lines 2488-2494

Item	Explanation
Source location	Lines 2488-2494 (7 source lines).
Purpose	Loads data from disk, network, browser storage, or a service.
Declaration	async function readPublishAuthCache() {
Touches	filesystem, authentication/security data
Calls detected	parse, readFile
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2488: async function readPublishAuthCache() {
2489:   try {
2490:     return JSON.parse(await readFile(publishAuthCachePath, "utf8"));
```

```
2491:   } catch {
2492:     return null;
2493:   }
2494: }
```

writePublishAuthCache - lines 2496-2535

Item	Explanation
Source location	Lines 2496-2535 (40 source lines).
Purpose	Persists data to local state, disk, credentials, or browser storage.
Declaration	async function writePublishAuthCache(access = {}) {
Touches	filesystem, authentication/security data
Calls detected	readPublishAuthCache, getTime, isArray, map, parsePublishAuthTimestamp, filter, isFinite, toISOString, publishAuthCacheScope, writeFile, stringify, chmod
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	3 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2496: async function writePublishAuthCache(access = {}) {
2497:   const previousCache = await readPublishAuthCache();
2498:   const checkedAt = new Date();
2499:   const checkedAtMs = checkedAt.getTime();
2500:   const sameTarget = previousCache
2501:     && previousCache.remote === access.remote
2502:     && previousCache.branch === access.branch
2503:     && previousCache.repository === access.repository;
2504:   const previousHistory = sameTarget && Array.isArray(previousCache.successHistory)
2505:     ? previousCache.successHistory
2506:     : [];
2507:   const successTimestamps = [
2508:     ...previousHistory
2509:       .map((entry) => parsePublishAuthTimestamp(entry))
2510:       .filter((timestamp) => Number.isFinite(timestamp))
2511:       .filter((timestamp) => checkedAtMs - timestamp < publishAuthExtendedTtlMs),
2512:     checkedAtMs
2513:   ];
2514:   const successHistory = successTimestamps.map((timestamp) => new Date(timestamp).toISOString());
2515:   const successCountLastWeek = successTimestamps
2516:     .filter((timestamp) => checkedAtMs - timestamp < publishAuthHistoryWindowMs)
2517:     .length;
2518:   const extendedTrust = successCountLastWeek > publishAuthExtendedThreshold;
2519:   const ttlMs = extendedTrust ? publishAuthExtendedTtlMs : publishAuthCacheTtlMs;
2520:   const cache = {
2521:     branch: access.branch || "",
2522:     checkedAt: checkedAt.toISOString(),
2523:     expiresAt: new Date(checkedAtMs + ttlMs).toISOString(),
2524:     extendedTrust,
2525:     remote: access.remote || "",
2526:     repository: access.repository || "",
2527:     scope: publishAuthCacheScope(),
2528:     successCountLastWeek,
2529:     successHistory,
2530:     trustDays: extendedTrust ? 30 : 1
2531:   };
2532:   await writeFile(publishAuthCachePath, `${JSON.stringify(cache, null, 2)}\n`, "utf8");
2533:   await chmod(publishAuthCachePath, 0o600).catch(() => {});
2534:   return cache;
2535: }
```

publishAuthCacheScope - lines 2537-2552

Item	Explanation
Source location	Lines 2537-2552 (16 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function publishAuthCacheScope() {
Touches	authentication/security data

Item	Explanation
Calls detected	userInfo, createHash, update, hostname, join, digest
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2537: function publishAuthCacheScope() {
2538:   let username = process.env.USERNAME || process.env.USER || "";
2539:   try {
2540:     username = os.userInfo().username || username;
2541:   } catch {
2542:     // Environment username is a sufficient fallback for cache scoping.
2543:   }
2544:   return createHash("sha256")
2545:     .update([
2546:       os.hostname(),
2547:       username,
2548:       root,
2549:       portfolioRoot
2550:     ].join("\n"))
2551:     .digest("hex");
2552: }
```

parsePublishAuthTimestamp - lines 2554-2559

Item	Explanation
Source location	Lines 2554-2559 (6 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function parsePublishAuthTimestamp(value = "") {
Touches	filesystem, authentication/security data
Calls detected	parse, isFinite, replace
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2554: function parsePublishAuthTimestamp(value = "") {
2555:   const direct = Date.parse(String(value || ""));
2556:   if (Number.isFinite(direct)) return direct;
2557:   const normalized = String(value || "").replace(/(\.\d{3})\d+(Z|[+-]\d{2}:?\d{2})$/i, "$1$2");
2558:   return Date.parse(normalized);
2559: }
```

publishAuthCacheExpiresAt - lines 2561-2567

Item	Explanation
Source location	Lines 2561-2567 (7 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function publishAuthCacheExpiresAt(cache) {
Touches	authentication/security data
Calls detected	parsePublishAuthTimestamp, isFinite, toISOString
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.

Item	Explanation
Async/return clues	0 await expression(s), 3 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2561: function publishAuthCacheExpiresAt(cache) {
2562:   const explicitExpiresAt = parsePublishAuthTimestamp(cache?.expiresAt || "");
2563:   if (Number.isFinite(explicitExpiresAt)) return new Date(explicitExpiresAt).toISOString();
2564:   const checkedAt = parsePublishAuthTimestamp(cache?.checkedAt || "");
2565:   if (!Number.isFinite(checkedAt)) return "";
2566:   return new Date(checkedAt + publishAuthCacheTtlMs).toISOString();
2567: }
```

publishAuthCacheIsFresh - lines 2569-2576

Item	Explanation
Source location	Lines 2569-2576 (8 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	function publishAuthCacheIsFresh(cache, access) {
Touches	authentication/security data
Calls detected	publishAuthCacheScope, parse, publishAuthCacheExpiresAt, isFinite, now
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 5 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2569: function publishAuthCacheIsFresh(cache, access) {
2570:   if (!cache || !access) return false;
2571:   if (cache.remote !== access.remote || cache.branch !== access.branch || cache.repository !==
access.repository) return false;
2572:   if (cache.scope !== publishAuthCacheScope()) return false;
2573:   const expiresAt = Date.parse(publishAuthCacheExpiresAt(cache));
2574:   if (!Number.isFinite(expiresAt)) return false;
2575:   return Date.now() < expiresAt;
2576: }
```

assertPublishAccess - lines 2578-2669

Item	Explanation
Source location	Lines 2578-2669 (92 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function assertPublishAccess(options = {}) {
Touches	authentication/security data
Calls detected	runPublishGit, trim, publishAccessError, gitFailureText, remoteUrlForDisplay, workspaceHasCompatibleSiteFiles, parseGitHubRemote, readPublishAuthCache, publishAuthCacheIsFresh, publishAuthCacheExpiresAt, synchronizePublishBranchFromRemote, ensurePublishHeadForWriteCheck
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	9 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2578: async function assertPublishAccess(options = {}) {
2579:   let insideWorkTree = "";
2580:   try {
```

```
    insideWorkTree = (await runPublishGit(["rev-parse", "--is-inside-work-tree"])).stdout.trim();
  } catch (error) {
    throw publishAccessError(
      "This workspace is not connected to a Git repository.",
      gitFailureText(error),
      { repository: defaultSiteRepository }
    );
  }

  if (insideWorkTree !== "true") {
    throw publishAccessError(
      "This workspace is local-only and cannot publish yet.",
      "Clone or associate a GitHub Pages/static-site repository before applying to site.",
      { repository: defaultSiteRepository }
    );
  }

  let remote = "";
  let branch = "";
  try {
    remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
    branch = (await runPublishGit(["branch", "--show-current"])).stdout.trim();
  } catch (error) {
    throw publishAccessError(
      "A publish remote or branch is missing.",
      gitFailureText(error),
      { repository: defaultSiteRepository }
    );
  }

  if (!remote || !branch) {
    throw publishAccessError(
      "A publish remote or branch is missing.",
      "Set the origin remote and use a named branch before applying to site.",
      { remote: remoteUrlForDisplay(remote), branch }
    );
  }

  if (options.requireCompatible !== false && !await workspaceHasCompatibleSiteFiles()) {
    throw publishAccessError(
      "This repository does not look like a compatible static portfolio website.",
      "The workspace must include index.html, styles.css, and script.js before it can be used as a
publish target.",
      { remote: remoteUrlForDisplay(remote), branch }
    );
  }

  const parsedRemote = parseGitHubRemote(remote);
  const repository = parsedRemote ? `${parsedRemote.owner}/${parsedRemote.repo}` :
remoteUrlForDisplay(remote);

  const access = {
    branch,
    remote: remoteUrlForDisplay(remote),
    repository,
    authorizationChecked: false
  };

  const cached = await readPublishAuthCache();
  if (!options.force && publishAuthCacheIsFresh(cached, access)) {
    return {
      ...access,
      authorizationCached: true,
      authorizationChecked: true,
      checkedAt: cached.checkedAt,
      expiresAt: publishAuthCacheExpiresAt(cached),
      extendedTrust: Boolean(cached.extendedTrust),
      successCountLastWeek: cached.successCountLastWeek || 0,
      trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1)
    };
  }

  const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
  const localHead = await ensurePublishHeadForWriteCheck();
  const writeCheck = await verifyPublishWriteAccess(access);

  const authCache = await writePublishAuthCache(access);
  return {
    ...access,
    remoteSync,
    localHead,
    writeCheck,
    authorizationChecked: true,
    authorizationCached: false,
    checkedAt: authCache.checkedAt,
    expiresAt: authCache.expiresAt,
    extendedTrust: Boolean(authCache.extendedTrust),
    successCountLastWeek: authCache.successCountLastWeek || 0,
    trustDays: authCache.trustDays || 1
  };
}
```

syncFromPublishTarget - lines 2671-2745

Item	Explanation
Source location	Lines 2671-2745 (75 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function syncFromPublishTarget() {
Touches	filesystem
Calls detected	assertPublishAccess, runPublishGit, trim, mkdtemp, join, tmpdir, runGit, fsAccess, push, includes, publishAccessError, resolveInsideRoot
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	14 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2671: async function syncFromPublishTarget() {
2672:   const publishAccess = await assertPublishAccess({ requireCompatible: false });
2673:   const branch = publishAccess.branch || "main";
2674:   const remote = (await runPublishGit(["remote", "get-url", "origin"])).stdout.trim();
2675:   const importRoot = await mkdtemp(path.join(os.tmpdir(), "omb-publish-target-"));
2676:   const cloneRoot = path.join(importRoot, "target");
2677:   const importPaths = [
2678:     "projects.json",
2679:     "assets",
2680:     "docs",
2681:     "Backgrounds",
2682:     "CNAME",
2683:     ".nojekyll",
2684:     "robots.txt",
2685:     "sitemap.xml"
2686:   ];
2687:
2688:   try {
2689:     await runGit(["clone", "--depth", "1", "--branch", branch, remote, cloneRoot], {
2690:       cwd: importRoot,
2691:       timeout: 3 * 60 * 1000
2692:     });
2693:
2694:     const availablePaths = [];
2695:     for (const importPath of importPaths) {
2696:       try {
2697:         await fsAccess(path.join(cloneRoot, importPath));
2698:         availablePaths.push(importPath);
2699:       } catch {
2700:         // Optional path is absent in the selected site repository.
2701:       }
2702:     }
2703:
2704:     if (!availablePaths.includes("projects.json")) {
2705:       throw publishAccessError(
2706:         "The selected repository does not contain a projects.json portfolio catalog.",
2707:         "The target repository must contain a compatible OMB Portfolio Builder catalog before it can be
imported.",
2708:         publishAccess
2709:       );
2710:     }
2711:
2712:     const backupRoot = resolveInsideRoot(
2713:       ".omb-backups",
2714:       `load-target-${new Date().toISOString().replace(/[:.]/g, "-")}`
2715:     );
2716:     await mkdir(backupRoot, { recursive: true });
2717:
2718:     for (const importPath of availablePaths) {
2719:       const sourcePath = path.join(cloneRoot, importPath);
2720:       const targetPath = resolveInsideRoot(importPath);
2721:       try {
2722:         await fsAccess(targetPath);
2723:         await cp(targetPath, path.join(backupRoot, importPath), { recursive: true, force: true });
2724:         await rm(targetPath, { recursive: true, force: true });
2725:       } catch {
2726:         // Nothing local to back up for this path.
2727:       }
2728:       await cp(sourcePath, targetPath, { recursive: true, force: true });
2729:     }
2730:
2731:     const importedCatalog = await readFile(path.join(cloneRoot, "projects.json"), "utf8");
```

```
2732:     await writeFile(draftPath, importedCatalog, "utf8");
2733:     const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: false });
2734:
2735:     return {
2736:       ...publishAccess,
2737:       imported: availablePaths,
2738:       portfolioSync,
2739:       backup: path.relative(root, backupRoot),
2740:       draftUpdated: true
2741:     };
2742:   } finally {
2743:     await rm(importRoot, { recursive: true, force: true });
2744:   }
2745: }
```

authenticateGitHubForTarget - lines 2747-2843

Item	Explanation
Source location	Lines 2747-2843 (97 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function authenticateGitHubForTarget(options = {}) {
Touches	authentication/security data
Calls detected	call, validatePublishRemoteUrl, validateCustomDomain, validateCredentialPair, publishAccessError, getPublishTargetInfo, ensureGitRepository, capturePublishTargetState, setOriginRemote, getGitStatus, detectRemoteDefaultBranch, checkoutPublishBranch
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	19 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2747: async function authenticateGitHubForTarget(options = {}) {
2748:   const {
2749:     repositoryUrl = "",
2750:     customDomain = "",
2751:     authUsername = "",
2752:     authPassword = ""
2753:   } = options;
2754:   const customDomainProvided = Object.prototype.hasOwnProperty.call(options, "customDomain");
2755:   const remoteUrl = validatePublishRemoteUrl(repositoryUrl);
2756:   const domain = validateCustomDomain(customDomain);
2757:   const credentials = validateCredentialPair(authUsername, authPassword);
2758:   if (!remoteUrl) {
2759:     throw publishAccessError(
2760:       "A GitHub repository URL is required before authentication.",
2761:       "Enter the GitHub Pages or compatible static-site repository URL, then click Authenticate target.",
2762:       await getPublishTargetInfo()
2763:     );
2764:   }
2765:
2766:   await ensureGitRepository();
2767:   const snapshot = await capturePublishTargetState();
2768:
2769:   try {
2770:     await setOriginRemote(remoteUrl);
2771:     const status = await getGitStatus();
2772:     const preliminaryTarget = await getPublishTargetInfo();
2773:     if (!status.git.ok) {
2774:       throw publishAccessError(
2775:         "Git for Windows is required for publishing.",
2776:         "Install Git for Windows, then reopen the builder and authenticate again.",
2777:         { ...preliminaryTarget, system: status }
2778:       );
2779:     }
2780:
2781:     const targetBranch = await detectRemoteDefaultBranch(remoteUrl) || "main";
2782:     await checkoutPublishBranch(targetBranch);
2783:
2784:     const targetBeforeAuth = await getPublishTargetInfo();
2785:     const cached = await readPublishAuthCache();
2786:     const cachedAccess = {
2787:       branch: targetBeforeAuth.branch,
2788:       remote: targetBeforeAuth.remote,
2789:       repository: targetBeforeAuth.repository
2790:     };
2791:     if (!credentials.cleanUsername && publishAuthCacheIsFresh(cached, cachedAccess)) {
2792:       await writeTargetCustomDomain(domain, customDomainProvided);
```

```
2793:       const target = await getPublishTargetInfo();
2794:       return {
2795:         ...cachedAccess,
2796:         branch: target.branch || cachedAccess.branch,
2797:         authorizationCached: true,
2798:         authorizationChecked: true,
2799:         checkedAt: cached.checkedAt,
2800:         expiresAt: publishAuthCacheExpiresAt(cached),
2801:         extendedTrust: Boolean(cached.extendedTrust),
2802:         successCountLastWeek: cached.successCountLastWeek || 0,
2803:         trustDays: cached.trustDays || (cached.extendedTrust ? 30 : 1),
2804:         credentialsStored: false,
2805:         credentialUsername: "",
2806:         system: status,
2807:         target
2808:       };
2809:     }
2810:
2811:     if (!status.credentialManager.ok && !credentials.cleanUsername) {
2812:       throw publishAccessError(
2813:         "Git Credential Manager is required for guided GitHub sign-in.",
2814:         "Install or repair Git for Windows with Git Credential Manager enabled, or provide a GitHub
username and personal access token.",
2815:         { ...targetBeforeAuth, system: status }
2816:       );
2817:     }
2818:
2819:     const storedCredentials = await storeGitCredentials(remoteUrl, authUsername, authPassword);
2820:     if (!storedCredentials.stored) {
2821:       await runGit(["credential-manager", "github", "login"], {
2822:         cwd: portfolioRoot,
2823:         timeout: 5 * 60 * 1000,
2824:         windowsHide: false
2825:       });
2826:     }
2827:
2828:     const access = await assertPublishAccess({ force: true, requireCompatible: false });
2829:     await writeTargetCustomDomain(domain, customDomainProvided);
2830:     const target = await getPublishTargetInfo();
2831:     return {
2832:       ...access,
2833:       branch: target.branch || access.branch,
2834:       credentialsStored: storedCredentials.stored,
2835:       credentialUsername: storedCredentials.username || "",
2836:       system: await getGitStatus(),
2837:       target
2838:     };
2839:   } catch (error) {
2840:     await restorePublishTargetState(snapshot);
2841:     throw error;
2842:   }
2843: }
```

installGitForWindows - lines 2845-2868

Item	Explanation
Source location	Lines 2845-2868 (24 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function installGitForWindows() {
Touches	external processes, network/API requests, authentication/security data
Calls detected	runOptionalCommand, spawn, unref
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2845: async function installGitForWindows() {
2846:   const winget = await runOptionalCommand("winget", ["--version"]);
2847:   const downloadUrl = "https://git-scm.com/download/win";
2848:   if (!winget.ok) {
2849:     return {
2850:       launched: false,
2851:       downloadUrl,
2852:       message: "winget was not found. Open the Git for Windows download page and install Git with Git
Credential Manager enabled."
2853:     };
```

```
  }

  const child = spawn("winget", ["install", "--id", "Git.Git", "-e", "--source", "winget"], {
    detached: true,
    shell: true,
    stdio: "ignore",
    windowsHide: false
  });
  child.unref();
  return {
    launched: true,
    downloadUrl,
    message: "The Git for Windows installer was launched through winget. Follow the installer prompts, then reopen the builder."
  };
}
```

getUpdateInfo - lines 2870-2906

Item	Explanation
Source location	Lines 2870-2906 (37 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function getUpdateInfo() {
Touches	network/API requests
Calls detected	fetch, Error, json, replace, find, has, compareVersions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	2 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
async function getUpdateInfo() {
  const currentVersion = process.env.OMB_APP_VERSION || packageJson.version || "0.0.0";
  const owner = "otienomaurice";
  const repo = "omb-portfolio-builder";
  try {
    const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
      headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
    });
    if (!response.ok) throw new Error(`GitHub returned ${response.status}.`);
    const release = await response.json();
    const latestVersion = String(release.tag_name || "").replace(/^builder-v/i, "");
    const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
    const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
    const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
    return {
      ok: true,
      currentVersion,
      latestVersion,
      updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
      updateBlocked,
      blockedReason: updateBlocked
        ? `Builder ${latestVersion} is skipped because its installer can hang while running the old uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
        : "",
      releaseUrl: release.html_url || "",
      installerUrl: installer?.browser_download_url || "",
      installerName: installer?.name || "",
      portableUrl: portable?.browser_download_url || ""
    };
  } catch (error) {
    return {
      ok: false,
      currentVersion,
      latestVersion: currentVersion,
      updateAvailable: false,
      error: error.message || "Could not check for updates."
    };
  }
```

installer - lines 2881-2897

Item	Explanation
Source location	Lines 2881-2897 (17 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
Touches	Mostly local calculations or local UI state.
Calls detected	find, has, compareVersions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2881:     const installer = (release.assets || []).find((asset) => /Setup-.*\.exe$/i.test(asset.name));
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:         ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:         : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

portable - lines 2882-2897

Item	Explanation
Source location	Lines 2882-2897 (16 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
Touches	Mostly local calculations or local UI state.
Calls detected	find, has, compareVersions
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2882:     const portable = (release.assets || []).find((asset) => /Portable-.*\.exe$/i.test(asset.name));
2883:     const updateBlocked = blockedAppUpdateVersions.has(latestVersion);
2884:     return {
2885:       ok: true,
2886:       currentVersion,
2887:       latestVersion,
2888:       updateAvailable: !updateBlocked && compareVersions(latestVersion, currentVersion) > 0,
2889:       updateBlocked,
2890:       blockedReason: updateBlocked
2891:         ? `Builder ${latestVersion} is skipped because its installer can hang while running the old
uninstaller during in-app updates. Wait for builder 0.2.17 or newer, then run Update again.`
2892:         : "",
2893:       releaseUrl: release.html_url || "",
2894:       installerUrl: installer?.browser_download_url || "",
2895:       installerName: installer?.name || "",
2896:       portableUrl: portable?.browser_download_url || ""
2897:     };
```

getBuilderReleaseDownloadReport - lines 2909-2914

Item	Explanation
Source location	Lines 2909-2914 (6 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function getBuilderReleaseDownloadReport() {
Touches	network/API requests
Calls detected	fetch
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2909: async function getBuilderReleaseDownloadReport() {
2910:   const owner = "otienomaurice";
2911:   const repo = "omb-portfolio-builder";
2912:   const response = await fetch(`https://api.github.com/repos/${owner}/${repo}/releases/latest`, {
2913:     headers: { "Accept": "application/vnd.github+json", "User-Agent": "OMB-Portfolio-Builder" }
2914:   });
```

assets - lines 2917-2923

Item	Explanation
Source location	Lines 2917-2923 (7 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const assets = (release.assets || []).map((asset) => ({
Touches	Mostly local calculations or local UI state.
Calls detected	map
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2917:   const assets = (release.assets || []).map((asset) => ({
2918:     name: asset.name || "",
2919:     downloadCount: Number(asset.download_count || 0),
2920:     sizeBytes: Number(asset.size || 0),
2921:     updatedAt: asset.updated_at || "",
2922:     url: asset.browser_download_url || ""
2923:   }));
```

getSecurityReport - lines 2933-2977

Item	Explanation
Source location	Lines 2933-2977 (45 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function getSecurityReport() {
Touches	authentication/security data
Calls detected	allSettled, getBuilderReleaseDownloadReport, readPublishAuthCache, getPublishTargetInfo, toISOString, publishAuthCacheIsFresh, publishAuthCacheExpiresAt
API endpoints	None detected inside this function body.

Item	Explanation
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2933: async function getSecurityReport() {
2934:   const [downloadResult, authCacheResult, targetResult] = await Promise.allSettled([
2935:     getBuilderReleaseDownloadReport(),
2936:     readPublishAuthCache(),
2937:     getPublishTargetInfo()
2938:   ]);
2939:   const authCache = authCacheResult.status === "fulfilled" ? authCacheResult.value : null;
2940:   const target = targetResult.status === "fulfilled" ? targetResult.value : null;
2941:   return {
2942:     ok: true,
2943:     generatedAt: new Date().toISOString(),
2944:     appDownloads: downloadResult.status === "fulfilled"
2945:       ? { ok: true, ...downloadResult.value }
2946:       : { ok: false, error: downloadResult.reason?.message || "GitHub release download counts were not
available." },
2947:     publishingAuth: {
2948:       targetRepository: target?.repository || "",
2949:       branch: target?.branch || "",
2950:       authenticated: Boolean(authCache && publishAuthCacheIsFresh(authCache, {
2951:         remote: target?.remote || "",
2952:         repository: target?.repository || "",
2953:         branch: target?.branch || ""
2954:       })),
2955:       expiresAt: publishAuthCacheExpiresAt(authCache) || "",
2956:       trustDays: Number(authCache?.trustDays || 0),
2957:       successCountLastWeek: Number(authCache?.successCountLastWeek || 0)
2958:     },
2959:     websiteAccess: {
2960:       available: false,
2961:       summary: "Visitor counts and visitor IP addresses are not available from a static GitHub Pages site
or GitHub release download counts.",
2962:       recommendedSetup: [
2963:         "Proxy the custom domain through Cloudflare instead of DNS-only.",
2964:         "Enable Cloudflare Web Analytics for aggregate page-view counts.",
2965:         "Use Cloudflare Logpush, Analytics Engine, or a Worker-backed endpoint if you need IP-level
security logs.",
2966:         "Publish a privacy notice before collecting raw IP addresses or persistent identifiers."
2967:       ]
2968:     },
2969:     securityControls: [
2970:       "Local builder write APIs are restricted to local requests.",
2971:       "Publishing target details are only returned to the local machine.",
2972:       "GitHub publishing authorization is cached for one day by default, or longer after repeated
successful authorizations.",
2973:       "The desktop app runs with Electron nodeIntegration disabled, contextIsolation enabled, and sandbox
enabled.",
2974:       "Deployable website security headers and security.txt metadata are included in the publishable site
files."
2975:     ]
2976:   };
2977: }
```

safeUpdateFileSegment - lines 2979-2984

Item	Explanation
Source location	Lines 2979-2984 (6 source lines).
Purpose	Validates or normalizes input so later code receives safe predictable values.
Declaration	function safeUpdateFileSegment(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	replace, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2979: function safeUpdateFileSegment(value = "") {
2980:   return String(value || "")
2981:     .replace(/[^a-z0-9._-]+/gi, "-")
2982:     .replace(/^-+|-+$/g, "")
2983:     .slice(0, 80) || "latest";
2984: }
```

powershellSingleQuoted - lines 2986-2988

Item	Explanation
Source location	Lines 2986-2988 (3 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	function powershellSingleQuoted(value = "") {
Touches	Mostly local calculations or local UI state.
Calls detected	replaceAll
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2986: function powershellSingleQuoted(value = "") {
2987:   return `'${String(value).replaceAll("'", "''")}'`;
2988: }
```

downloadAndLaunchAppUpdate - lines 2990-3165

Item	Explanation
Source location	Lines 2990-3165 (176 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	async function downloadAndLaunchAppUpdate() {
Touches	filesystem, external processes, network/API requests
Calls detected	getUpdateInfo, Error, join, tmpdir, mkdir, safeUpdateFileSegment, fetch, from, arrayBuffer, writeFile, dirname, homedir
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	7 await expression(s), 6 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
2990: async function downloadAndLaunchAppUpdate() {
2991:   const update = await getUpdateInfo();
2992:   if (!update.ok) throw new Error(update.error || "Could not check for updates.");
2993:   if (update.updateBlocked) {
2994:     throw new Error(update.blockedReason || `Builder ${update.latestVersion} is temporarily blocked from
automatic updates.`);
2995:   }
2996:   if (!update.updateAvailable) {
2997:     throw new Error(`OMB Portfolio Builder ${update.currentVersion || ""} is already up to date.`);
2998:   }
2999:   if (!update.installerUrl) {
3000:     throw new Error("The latest GitHub Release does not include a Windows setup installer.");
3001:   }
3002:   if (process.platform !== "win32") {
3003:     throw new Error("Automatic installer updates are currently available only on Windows.");
3004:   }
3005:
3006:   const updateRoot = path.join(os.tmpdir(), "omb-portfolio-builder-updates");
3007:   await mkdir(updateRoot, { recursive: true });
3008:   const installerName = update.installerName && /\.exe$/i.test(update.installerName)
3009:     ? update.installerName
```

```
    : `OMB-Portfolio-Builder-Setup-${safeUpdateFileSegment(update.latestVersion)}-x64.exe`;
  const installerPath = path.join(updateRoot, installerName);

  const response = await fetch(update.installerUrl, {
    headers: { "Accept": "application/octet-stream", "User-Agent": "OMB-Portfolio-Builder" }
  });
  if (!response.ok) throw new Error(`GitHub returned ${response.status} while downloading the installer.`);
  const installerBuffer = Buffer.from(await response.arrayBuffer());
  if (installerBuffer.length < 1024 * 1024) {
    throw new Error("The downloaded installer was unexpectedly small, so the update was stopped.");
  }
  await writeFile(installerPath, installerBuffer);

  const updateLogPath = path.join(updateRoot, `update-${safeUpdateFileSegment(update.latestVersion)}.log`);
  const currentExecutable = /OMB Portfolio Builder\.exe$/i.test(process.execPath || "") ? process.execPath : "";
  const currentInstallDirectory = currentExecutable ? path.dirname(currentExecutable) : "";
  const localAppDataExecutable = process.env.LOCALAPPDATA
    ? path.join(process.env.LOCALAPPDATA, "Programs", "OMB Portfolio Builder", "OMB Portfolio Builder.exe")
    : "";
  const legacyManagedApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder", "OMB Portfolio Builder.exe");
  const legacyFlatApplicationExecutable = path.join(os.homedir(), "OMB", "application", "OMB Portfolio Builder.exe");
  const relaunchCandidates = Array.from(new Set([
    localAppDataExecutable,
    currentExecutable,
    currentInstallDirectory ? path.join(currentInstallDirectory, "OMB Portfolio Builder.exe") : "",
    legacyFlatApplicationExecutable,
    legacyManagedApplicationExecutable,
    process.env.ProgramFiles ? path.join(process.env.ProgramFiles, "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : "",
    process.env["ProgramFiles(x86)"] ? path.join(process.env["ProgramFiles(x86)"], "OMB Portfolio Builder", "OMB Portfolio Builder.exe") : ""
  ].filter(Boolean)));
  const relaunchCandidatesPs = `@(${relaunchCandidates.map(powershellSingleQuoted).join(", ")})`;
  const launcherPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.ps1`);
  const launcherCommandPath = path.join(updateRoot, `run-update-${safeUpdateFileSegment(update.latestVersion)}.cmd`);
  const launcherScript = [
    "$ErrorActionPreference = 'Continue'",
    `$parentPid = ${process.pid}`,
    `$installer = ${powershellSingleQuoted(installerPath)}`,
    `$log = ${powershellSingleQuoted(updateLogPath)}`,
    `$candidates = ${relaunchCandidatesPs}`,
    "function Write-UpdateLog([string]$Message) {",
    "  try { Add-Content -LiteralPath $log -Value ((Get-Date).ToString('s') + ' ' + $Message) } catch {}",
    "}",
    "function Wait-ForExecutable([string[]]$Paths, [int]$Seconds) {",
    "  $deadline = (Get-Date).AddSeconds($Seconds)",
    "  do {",
    "    foreach ($candidate in $Paths) {",
    "      if ($candidate -and (Test-Path -LiteralPath $candidate)) { return $candidate }",
    "    }",
    "    Start-Sleep -Seconds 1",
    "  } while ((Get-Date) -lt $deadline)",
    "  return $null",
    "}",
    "function Stop-BuilderProcesses([int]$ProcessId, [string[]]$Paths) {",
    "  try {",
    "    $normalizedPaths = @($Paths | Where-Object { $_ } | ForEach-Object {",
    "      try { [System.IO.Path]::GetFullPath($_).ToLowerInvariant() } catch { $_.ToLowerInvariant() }",
    "    })",
    "    $processes = Get-CimInstance Win32_Process | Where-Object {",
    "      $_.ProcessId -eq $ProcessId -or",
    "      $_.Name -eq 'OMB Portfolio Builder.exe' -or",
    "      ($_.ExecutablePath -and ($normalizedPaths -contains ([System.IO.Path]::GetFullPath($_.ExecutablePath).ToLowerInvariant())))",
    "    } | Sort-Object ProcessId -Unique",
    "    foreach ($process in $processes) {",
    "      try {",
    "        Write-UpdateLog ('Stopping previous builder process PID ' + $process.ProcessId + ' at ' + $process.ExecutablePath)",
    "        Stop-Process -Id $process.ProcessId -Force -ErrorAction SilentlyContinue",
    "      } catch { Write-UpdateLog ('Could not stop PID ' + $process.ProcessId + ': ' + $_.Exception.Message) }",
    "    }",
    "    Start-Sleep -Seconds 2",
    "  } catch { Write-UpdateLog ('Could not stop previous builder processes: ' + $_.Exception.Message) }",
    "}",
    "function Run-Installer([switch]$Elevated) {",
    "  $process = $null",
    "  if ($Elevated) {",
    "    Write-UpdateLog 'Starting installer with Windows elevation prompt.'",
    "    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -Verb RunAs -PassThru",
    "  } else {",
```

```
 3087:     "    Write-UpdateLog 'Starting installer silently.'",
 3088:     "    $process = Start-Process -FilePath $installer -ArgumentList @('/S') -WindowStyle Hidden
-PassThru",
 3089:     "  }",
 3090:     "  if (-not $process) { return 0 }",
 3091:     "  if (-not $process.WaitForExit(900000)) {",
 3092:     "    Write-UpdateLog ('Installer timed out after 15 minutes. PID: ' + $process.Id)",
 3093:     "    try { Stop-Process -Id $process.Id -Force -ErrorAction SilentlyContinue } catch { Write-
UpdateLog ('Could not stop timed-out installer: ' + $_.Exception.Message) }",
 3094:     "    return 124",
 3095:     "  }",
 3096:     "  return $process.ExitCode",
 3097:     "}",
 3098:     "Write-UpdateLog 'Update launcher started.'",
 3099:     "Start-Sleep -Seconds 3",
 3100:     "try { Wait-Process -Id $parentPid -Timeout 8; Write-UpdateLog 'Previous app process exited.' } catch
 { Write-UpdateLog ('Previous app wait finished: ' + $_.Exception.Message) }",
 3101:     "Stop-BuilderProcesses $parentPid $candidates",
 3102:     "Start-Sleep -Seconds 1",
 3103:     "$installerProcess = $null",
 3104:     "$installerExit = 1",
 3105:     "try {",
 3106:     "  Write-UpdateLog ('Starting installer: ' + $installer)",
 3107:     "  $installerExit = Run-Installer",
 3108:     "  Write-UpdateLog ('Installer exit code: ' + $installerExit)",
 3109:     "} catch {",
      ... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

syncPortfolioPublishFiles - lines 3167-3200

Item	Explanation
Source location	Lines 3167-3200 (34 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function syncPortfolioPublishFiles(options = {}) {
Touches	filesystem
Calls detected	samePath, mkdir, resolveInsideRoot, resolveInsidePortfolioRoot, pathExists, rm, dirname, cp, push
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	6 await expression(s), 2 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3167: async function syncPortfolioPublishFiles(options = {}) {
3168:   if (samePath(root, portfolioRoot)) {
3169:     return { copied: [], skipped: [], workspace: portfolioRoot, separatedWorkspace: false };
3170:   }
3171:
3172:   const { removeMissing = false } = options;
3173:   const copied = [];
3174:   const skipped = [];
3175:
3176:   await mkdir(portfolioRoot, { recursive: true });
3177:
3178:   for (const relativePath of publishPaths) {
3179:     const sourcePath = resolveInsideRoot(relativePath);
3180:     const targetPath = resolveInsidePortfolioRoot(relativePath);
3181:     if (await pathExists(sourcePath)) {
3182:       await rm(targetPath, { recursive: true, force: true });
3183:       await mkdir(path.dirname(targetPath), { recursive: true });
3184:       await cp(sourcePath, targetPath, { recursive: true, force: true });
3185:       copied.push(relativePath);
3186:     } else if (removeMissing) {
3187:       await rm(targetPath, { recursive: true, force: true });
3188:       skipped.push(relativePath);
3189:     } else {
3190:       skipped.push(relativePath);
3191:     }
3192:   }
3193:
3194:   return {
3195:     copied,
3196:     skipped,
3197:     workspace: portfolioRoot,
3198:     separatedWorkspace: true
3199:   };
3200: }
```

publishSiteChanges - lines 3202-3238

Item	Explanation
Source location	Lines 3202-3238 (37 source lines).
Purpose	Part of publishing, Git, target repository, or authorization behavior.
Declaration	async function publishSiteChanges(publishAccess = null) {
Touches	Mostly local calculations or local UI state.
Calls detected	assertPublishAccess, runPublishGit, trim, synchronizePublishBranchFromRemote, syncPortfolioPublishFiles, bumpPublishedAssetVersions, stageablePublishPaths, Error, toISOString, slice
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	10 await expression(s), 1 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3202: async function publishSiteChanges(publishAccess = null) {
3203:   const access = publishAccess || await assertPublishAccess();
3204:   const branch = access.branch || (await runPublishGit(["branch", "--show-current"])).stdout.trim();
3205:   const remoteSync = await synchronizePublishBranchFromRemote(branch, access);
3206:   const portfolioSync = await syncPortfolioPublishFiles({ removeMissing: true });
3207:   await bumpPublishedAssetVersions(portfolioRoot);
3208:
3209:   const stageablePaths = await stageablePublishPaths(publishPaths);
3210:   if (!stageablePaths.length) {
3211:     throw new Error("No compatible site files were available to publish.");
3212:   }
3213:
3214:   await runPublishGit(["add", "-A", "--", ...stageablePaths]);
3215:   const status = await runPublishGit(["status", "--porcelain", "--", ...stageablePaths]);
3216:   const hasChanges = status.stdout.trim().length > 0;
3217:
3218:   let commit = null;
3219:   if (hasChanges) {
3220:     const message = `Update portfolio site ${new Date().toISOString().slice(0, 10)}`;
3221:     commit = await runPublishGit(["commit", "-m", message]);
3222:   }
3223:
3224:   const pushArgs = branch ? ["push", "origin", branch] : ["push"];
3225:   const push = await runPublishGit(pushArgs);
3226:
3227:   return {
3228:     ...access,
3229:     portfolioWorkspace: portfolioRoot,
3230:     remoteSync,
3231:     portfolioSync,
3232:     branch: branch || "current branch",
3233:     committed: hasChanges,
3234:     commitOutput: commit?.stdout || commit?.stderr || "",
3235:     pushed: true,
3236:     pushOutput: push.stdout || push.stderr || ""
3237:   };
3238: }
```

handlePortfolioAi - lines 3240-3322

Item	Explanation
Source location	Lines 3240-3322 (83 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handlePortfolioAi(request, response) {
Touches	network/API requests, authentication/security data
Calls detected	readRequestJson, clampText, trim, cleanConversationHistory, Set, has, sendJson, enrichPortfolioContext, callOllamaPortfolioAi, toLowerCase, stringify, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	5 await expression(s), 4 return statement(s).

Item	Explanation
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3240: async function handlePortfolioAi(request, response) {
3241:   const body = await readRequestJson(request);
3242:   const question = clampText(body.question, 1200).trim();
3243:   const conversation = cleanConversationHistory(body.conversation);
3244:   const validIntents = new Set(["general_conversation", "general_engineering", "general_knowledge",
"portfolio_specific"]);
3245:   const intent = validIntents.has(body.intent) ? body.intent : "portfolio_specific";
3246:   const allowWebSearch = body.allowWebSearch === true;
3247:
3248:   if (!question) {
3249:     sendJson(response, 400, { error: "Question is required." });
3250:     return;
3251:   }
3252:
3253:   const context = await enrichPortfolioContext({ ...(body.context || {}), question });
3254:
3255:   const apiKey = process.env.OPENAI_API_KEY || "";
3256:   if (!apiKey) {
3257:     const ollama = await callOllamaPortfolioAi({ question, intent, conversation, context, allowWebSearch
});
3258:     if (ollama.ok) {
3259:       sendJson(response, 200, {
3260:         answer: ollama.answer,
3261:         model: ollama.model,
3262:         provider: "ollama",
3263:         usedWebSearch: false
3264:       });
3265:       return;
3266:     }
3267:     sendJson(response, 503, { error: ollama.error || "No local Ollama model or OPENAI_API_KEY is
available for the local backend." });
3268:     return;
3269:   }
3270:
3271:   const model = process.env.OPENAI_MODEL || "gpt-5.4";
3272:   const fallbackModel = process.env.OPENAI_FALLBACK_MODEL || "gpt-4.1";
3273:   const webSearchMode = String(process.env.OPENAI_ENABLE_WEB_SEARCH || "auto").toLowerCase();
3274:   const enableWebSearch = webSearchMode === "true" || (webSearchMode !== "false" && allowWebSearch);
3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         }]
3298:       }
3299:     ],
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });
3304:
3305:   const callModel = async (selectedModel) => {
3306:     const payload = buildPayload(selectedModel);
3307:     if (enableWebSearch) {
3308:       payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
3309:     }
3310:
3311:     const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
3312:       method: "POST",
3313:       headers: {
3314:         "Authorization": `Bearer ${apiKey}`,
3315:         "Content-Type": "application/json"
3316:       },
3317:       body: JSON.stringify(payload)
3318:     });
```

```
3319:
3320:     const data = await openAiResponse.json().catch(() => ({}));
3321:     return { data, openAiResponse, selectedModel };
3322:   };
```

buildPayload - lines 3275-3303

Item	Explanation
Source location	Lines 3275-3303 (29 source lines).
Purpose	Creates an object, UI structure, process, payload, or generated artifact.
Declaration	const buildPayload = (selectedModel) => ({
Touches	authentication/security data
Calls detected	clampText, stringify, join
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	0 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
3275:   const buildPayload = (selectedModel) => ({
3276:     model: selectedModel,
3277:     input: [
3278:       {
3279:         role: "developer",
3280:         content: [{ type: "input_text", text: portfolioAiInstructions }]
3281:       },
3282:       {
3283:         role: "user",
3284:         content: [{
3285:           type: "input_text",
3286:           text: [
3287:             `Visitor question: ${question}`,
3288:             `Question intent: ${intent}`,
3289:             `Web search allowed for this question: ${enableWebSearch ? "yes" : "no"}`,
3290:             "",
3291:             "Recent conversation JSON:",
3292:             clampText(JSON.stringify(conversation, null, 2), 8000),
3293:             "",
3294:             "Portfolio context JSON:",
3295:             clampText(JSON.stringify(context, null, 2), 18000)
3296:           ].join("\n")
3297:         }]
3298:       }
3299:     ],
3300:     max_output_tokens: Number(process.env.OPENAI_MAX_OUTPUT_TOKENS || 1100),
3301:     reasoning: { effort: process.env.OPENAI_REASONING_EFFORT || "medium" },
3302:     text: { verbosity: process.env.OPENAI_VERBOSITY || "medium" }
3303:   });
```

callModel - lines 3305-3318

Item	Explanation
Source location	Lines 3305-3318 (14 source lines).
Purpose	Part of the local backend API, filesystem, compiler, Git, AI, update, or publish workflow.
Declaration	const callModel = async (selectedModel) => {
Touches	network/API requests, authentication/security data
Calls detected	async, buildPayload, fetch, stringify
API endpoints	None detected inside this function body.
Storage keys	No local/session storage keys detected.
Async/return clues	1 await expression(s), 0 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
  const callModel = async (selectedModel) => {
    const payload = buildPayload(selectedModel);
    if (enableWebSearch) {
      payload.tools = [{ type: "web_search", search_context_size:
process.env.OPENAI_WEB_SEARCH_CONTEXT_SIZE || "low" }];
    }

    const openAiResponse = await fetch("https://api.openai.com/v1/responses", {
      method: "POST",
      headers: {
        "Authorization": `Bearer ${apiKey}`,
        "Content-Type": "application/json"
      },
      body: JSON.stringify(payload)
    });
```

handleApi - lines 3342-3602

Item	Explanation
Source location	Lines 3342-3602 (261 source lines).
Purpose	Handles an event, request, command, or user action.
Declaration	async function handleApi(request, response, url) {
Touches	filesystem, network/API requests, authentication/security data
Calls detected	readJsonFile, sendJson, join, isLocalRequest, getPublishTargetInfo, getGitStatus, getUpdateInfo, getSecurityReport, compileToolStatus, handlePortfolioAi, readRequestJson, normalizeCodeLanguage
API endpoints	/api/app-update, /api/app-update/install, /api/apply-catalog, /api/catalog, /api/code/beautify, /api/code/compile, /api/code/install-tools, /api/code/save, /api/code/tools, /api/github-authenticate, /api/install-git, /api/portfolio-ai, /api/publish-target, /api/save-draft, /api/security-report, /api/sync-from-publish-target, /api/system-check, /api/templates, /api/upload
Storage keys	No local/session storage keys detected.
Async/return clues	35 await expression(s), 29 return statement(s).
Debug approach	Trigger the user action that reaches this function, inspect inputs/state before the function, then inspect the returned value, DOM update, file write, process output, or API response after it runs.

Source excerpt:

```
async function handleApi(request, response, url) {
  if (request.method === "GET" && url.pathname === "/api/catalog") {
    try {
      const catalog = await readJsonFile(draftPath);
      sendJson(response, 200, { source: "draft", catalog });
    } catch {
      sendJson(response, 200, { source: "published", catalog: await readJsonFile(catalogPath) });
    }
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/templates") {
    sendJson(response, 200, await readJsonFile(path.join(root, "project-templates.json")));
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/publish-target") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Publishing target details are only available from this computer."
});
      return true;
    }
    sendJson(response, 200, { ok: true, target: await getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/system-check") {
    sendJson(response, 200, { ok: true, system: await getGitStatus(), target: await
getPublishTargetInfo() });
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/app-update") {
    sendJson(response, 200, await getUpdateInfo());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/security-report") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Security reports are only available from this computer." });
      return true;
    }
```

```
    sendJson(response, 200, await getSecurityReport());
    return true;
  }

  if (request.method === "GET" && url.pathname === "/api/code/tools") {
    if (!isLocalRequest(request)) {
      sendJson(response, 403, { error: "Compiler details are only available from this computer." });
      return true;
    }
    sendJson(response, 200, { ok: true, tools: await compileToolStatus() });
    return true;
  }

  if (request.method !== "POST") return false;

  if (!isLocalRequest(request)) {
    sendJson(response, 403, { error: "Write actions are only allowed from this computer." });
    return true;
  }

  if (url.pathname === "/api/portfolio-ai") {
    await handlePortfolioAi(request, response);
    return true;
  }

  if (url.pathname === "/api/code/beautify") {
    try {
      const body = await readRequestJson(request);
      const language = normalizeCodeLanguage(body.language || detectCodeLanguageFromSource(body.code,
body.fileName));
      sendJson(response, 200, {
        ok: true,
        language,
        code: beautifyCode(body.code || "", language)
      });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be beautified." });
    }
    return true;
  }

  if (url.pathname === "/api/code/save") {
    try {
      const body = await readRequestJson(request);
      const saved = await saveCompileSource(body);
      sendJson(response, 200, { ok: true, saved });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be saved." });
    }
    return true;
  }

  if (url.pathname === "/api/code/compile") {
    try {
      const body = await readRequestJson(request);
      const result = await compileAndRunCode(body);
      sendJson(response, 200, { ok: result.ok, result });
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Code could not be compiled.", result:
null });
    }
    return true;
  }

  if (url.pathname === "/api/code/install-tools") {
    try {
      const body = await readRequestJson(request);
      const result = await installCompilerTools(body.language || "");
      sendJson(response, 200, result);
    } catch (error) {
      sendJson(response, 400, { ok: false, error: error.message || "Compiler tools could not be
installed." });
    }
    return true;
  }

  if (url.pathname === "/api/publish-target") {
    await readRequestJson(request).catch(() => ({}));
    sendJson(response, 400, {
      ok: false,
      error: "Publishing target setup now requires GitHub authentication.",
      details: "Use Authenticate target. The builder keeps the previous target until GitHub write access
is verified."
    });
... excerpt truncated after 120 lines. Open the source file for the rest of this function.
```

Representative opening snippet

```
import { createServer } from "node:http";
import { execFile, spawn } from "node:child_process";
```

```
    3: import { createHash } from "node:crypto";
    4: import { access as fsAccess, chmod, cp, mkdir, mkdtemp, readFile, readdir, rm, writeFile } from
"node:fs/promises";
    5: import os from "node:os";
    6: import path from "node:path";
    7: import { promisify } from "node:util";
    8: import { fileURLToPath } from "node:url";
    9:
   10: const root = path.dirname(fileURLToPath(import.meta.url));
   11: const portfolioRoot = path.resolve(process.env.OMB_PORTFOLIO_WORKSPACE || root);
   12: const compileRoot = path.resolve(process.env.OMB_CODE_WORKSPACE || path.join(path.dirname(root),
"compile-code"));
   13: const port = Number(process.env.PORT || 8080);
   14: const host = process.env.HOST || "0.0.0.0";
   15: const execFileAsync = promisify(execFile);
   16: const packageJson = JSON.parse(await readFile(path.join(root, "package.json"), "utf8").catch(() =>
"{}"));
   17:
   18: const types = {
   19:   ".css": "text/css",
   20:   ".csv": "text/csv",
   21:   ".html": "text/html",
   22:   ".ico": "image/x-icon",
   23:   ".js": "application/javascript",
   24:   ".json": "application/json",
   25:   ".md": "text/markdown",
   26:   ".pdf": "application/pdf",
   27:   ".png": "image/png",
   28:   ".jpg": "image/jpeg",
   29:   ".jpeg": "image/jpeg",
   30:   ".svg": "image/svg+xml",
   31:   ".txt": "text/plain",
   32:   ".webp": "image/webp",
   33:   ".xml": "application/xml",
   34:   ".xlsx": "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
   35:   ".zip": "application/zip"
   36: };
   37:
   38: const draftPath = path.join(root, "projects.local.json");
   39: const catalogPath = path.join(root, "projects.json");
   40: const publishAuthCachePath = path.join(root, ".omb-publish-session.json");
   41: const publishPaths = [
   42:   "projects.json",
   43:   "docs",
   44:   "assets",
   45:   "index.html",
   46:   "styles.css",
   47:   "script.js",
   48:   "electronics-search.js",
   49:   "Backgrounds",
   50:   ".nojekyll",
   51:   "_headers",
   52:   ".well-known",
   53:   "robots.txt",
   54:   "sitemap.xml",
   55:   "CNAME"
   56: ];
   57: const publishAuthCacheTtlMs = 24 * 60 * 60 * 1000;
   58: const publishAuthExtendedTtlMs = 30 * 24 * 60 * 60 * 1000;
   59: const publishAuthHistoryWindowMs = 7 * 24 * 60 * 60 * 1000;
   60: const publishAuthExtendedThreshold = 3;
   61: const defaultSiteRepository = process.env.OMB_BUILDER_REPOSITORY || "";
   62: const blockedAppUpdateVersions = new Set(["0.2.16"]);
   63: const publishAuthorizationHelp = [
   64:   "Publishing was blocked before live website files were applied.",
   65:   "Open Publishing target, enter the repository URL, then click Authenticate target.",
   66:   "A GitHub/Git Credential Manager browser sign-in may open. Sign in with an account that has write
access to the selected Pages repository.",
   67:   "Authenticate target only verifies write access and remembers it for the matching repository and
branch.",
   68:   "Load from target only imports compatible website files from the authenticated repository into the
local builder workspace.",
   69:   "Until a compatible writable website repository is associated, the builder remains local-only."
   70: ].join(" ");
   71: const gitCandidates = [
   72:   process.env.GIT_EXE,
```